
Projeto de Extensão

Aula 4: O Cérebro do Robô - Do Caos à Solução



Cronograma

Todas as aulas serão das 13:30h às 17:30h

Aula 1: Apresentação e Introdução a Sensores
27/09 (sábado) e 01/10 (quarta)

Aula 2: Programação na linguagem C/C++
11/10 (sábado) e 22/10 (quarta)

Aula 3: Montagem de um sensor de baixo custo
25/10 (sábado) e 29/10 (quarta)

**Aula 4: Algoritmos,
comutadores e lógica do código**
08/11 (sábado) e 19/11 (quarta)

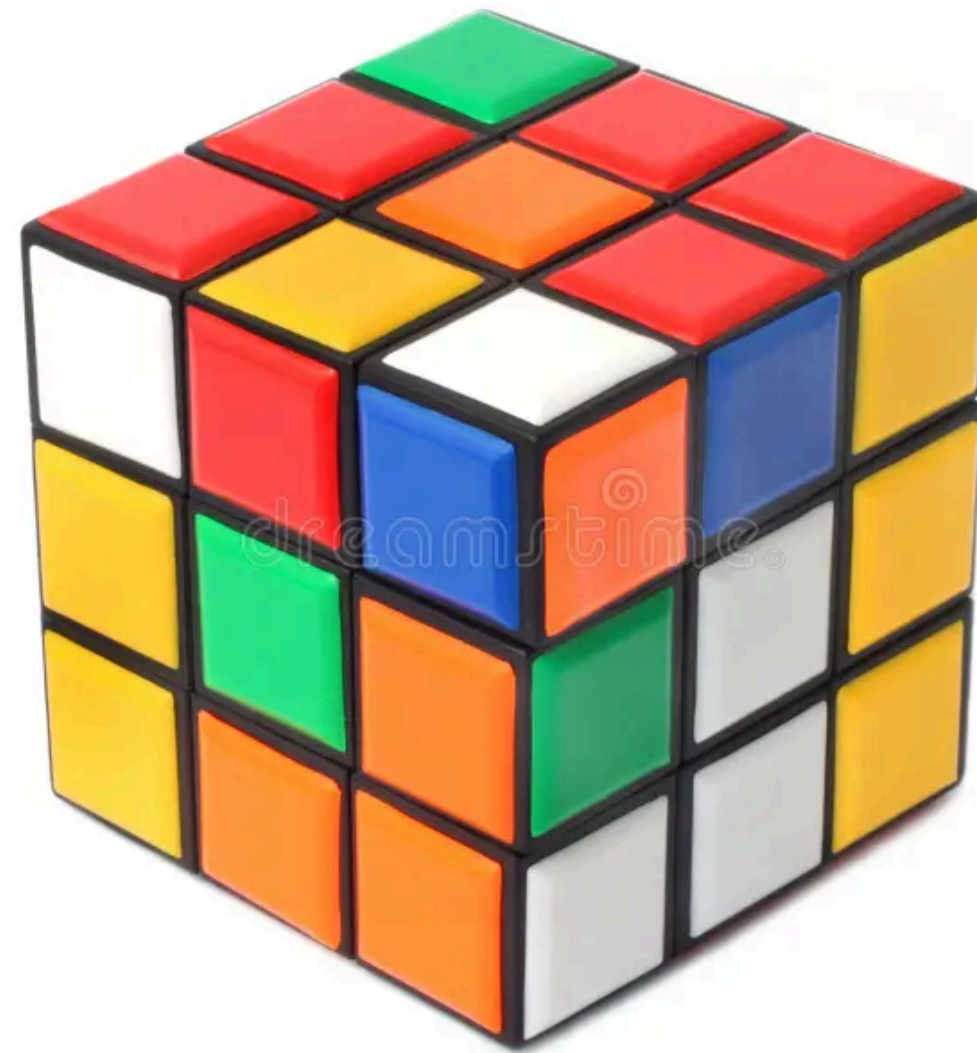
Aula 5: Motores e drivers
29/11 (sábado)

Aula 6: Visita a Feec
06/12 (sábado) e 03/12 (quarta)

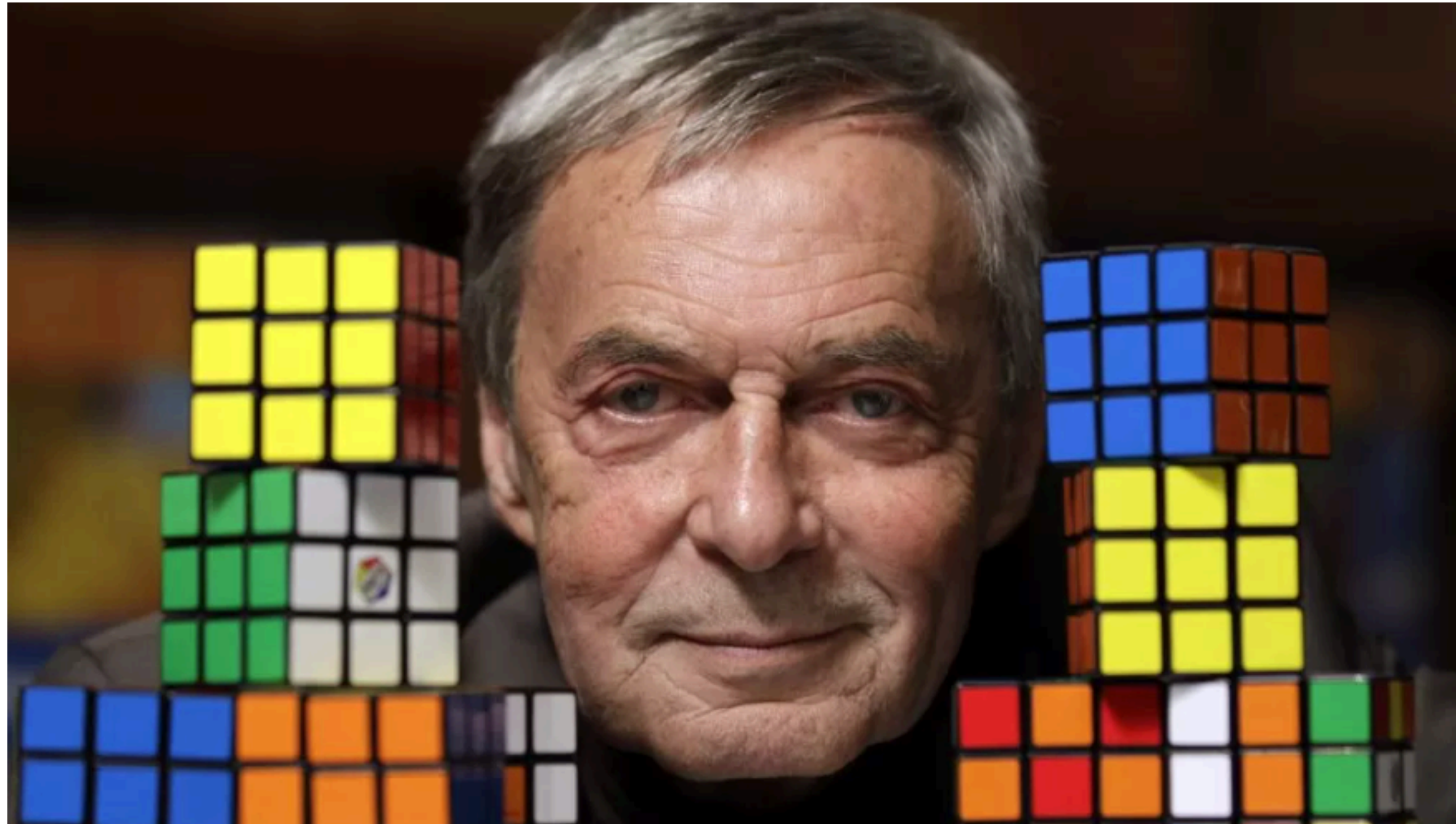
Resumo da Aula

- Introdução ao Cubo Mágico;
- Engenharia dos Sensores
- Protocolo de Input (Olhos → Cérebro);
- Métodos de Solução do Cubo Mágico;
- Lógica de Computadores
- Teste do código do algoritmo em Python ;
- Protocolo de Output (Cérebro → Braços)

Rubik's Cube (Magic Cube)

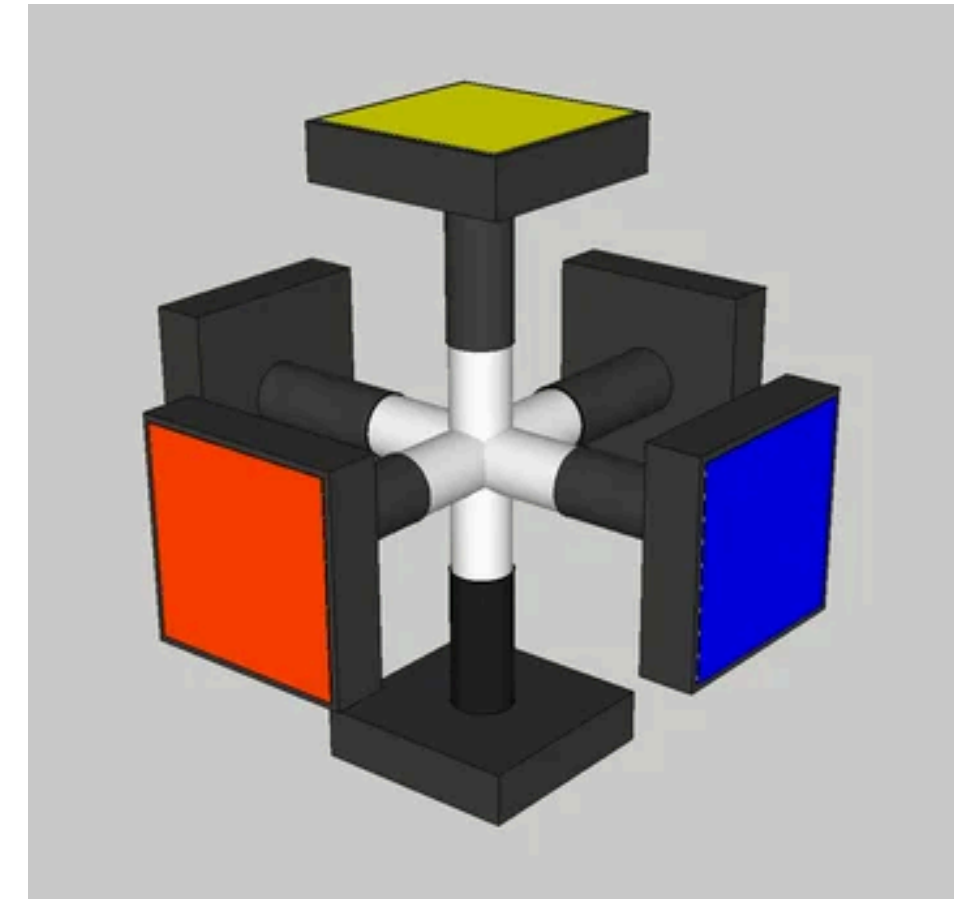
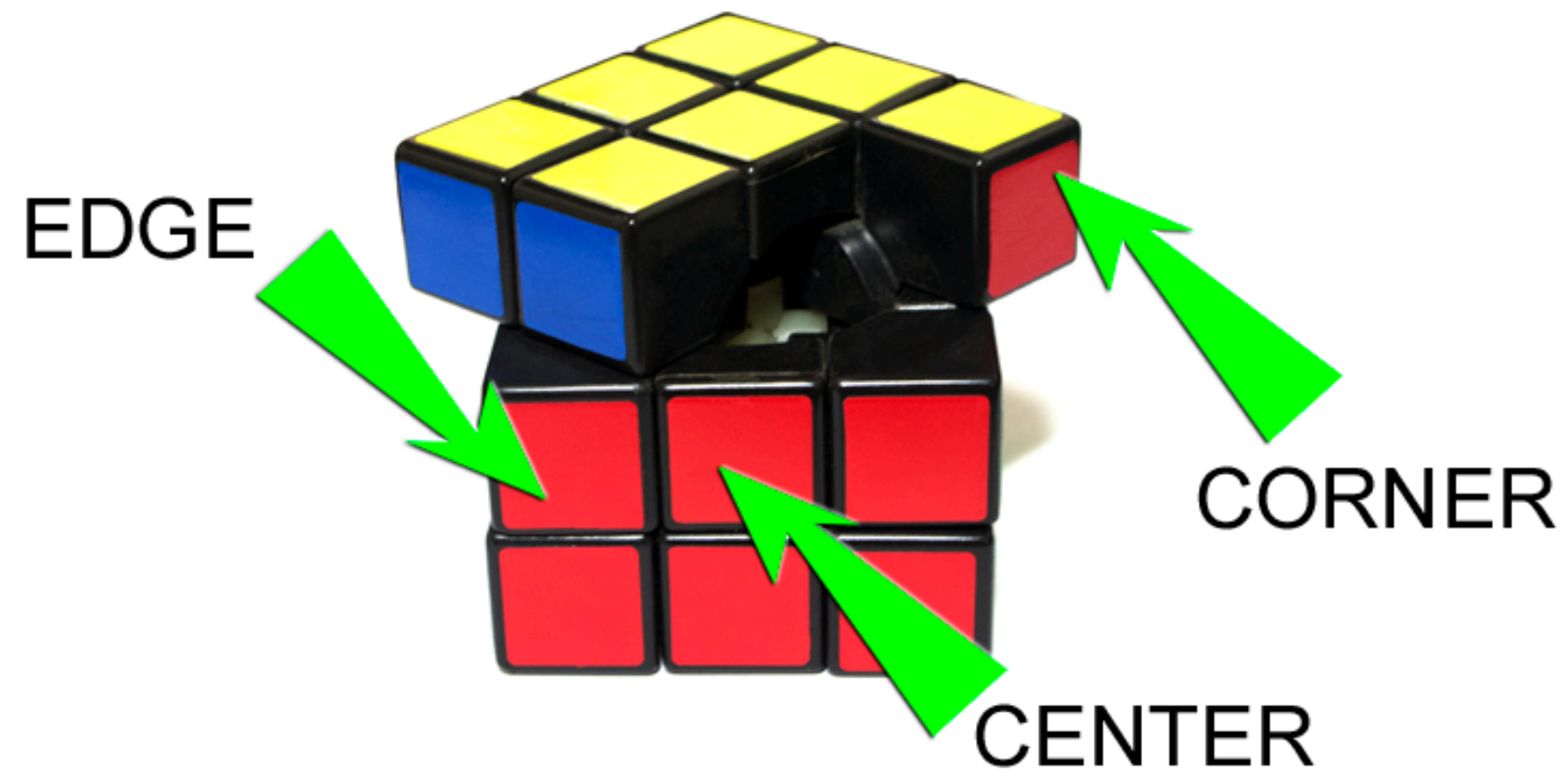


Ernő Rubik



Professor de arquitetura húngaro,
criou o cubo mágico em 1974.

Arquitetura do cubo



CORE

Ao todo temos 6 faces com 9 adesivos cada, totalizando 54 adesivos

Temos 6 centros, 8 pontas e 12 meios

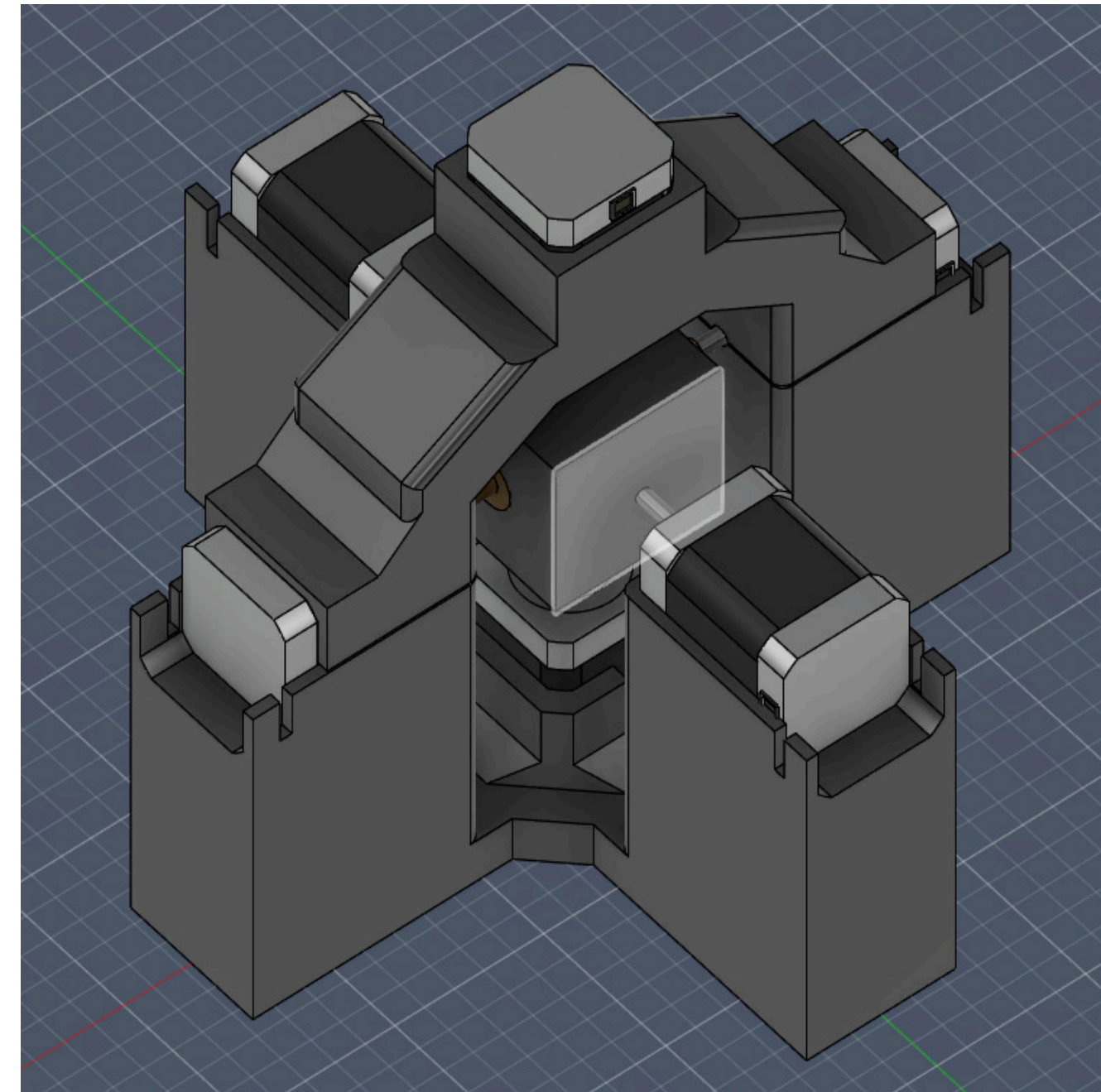
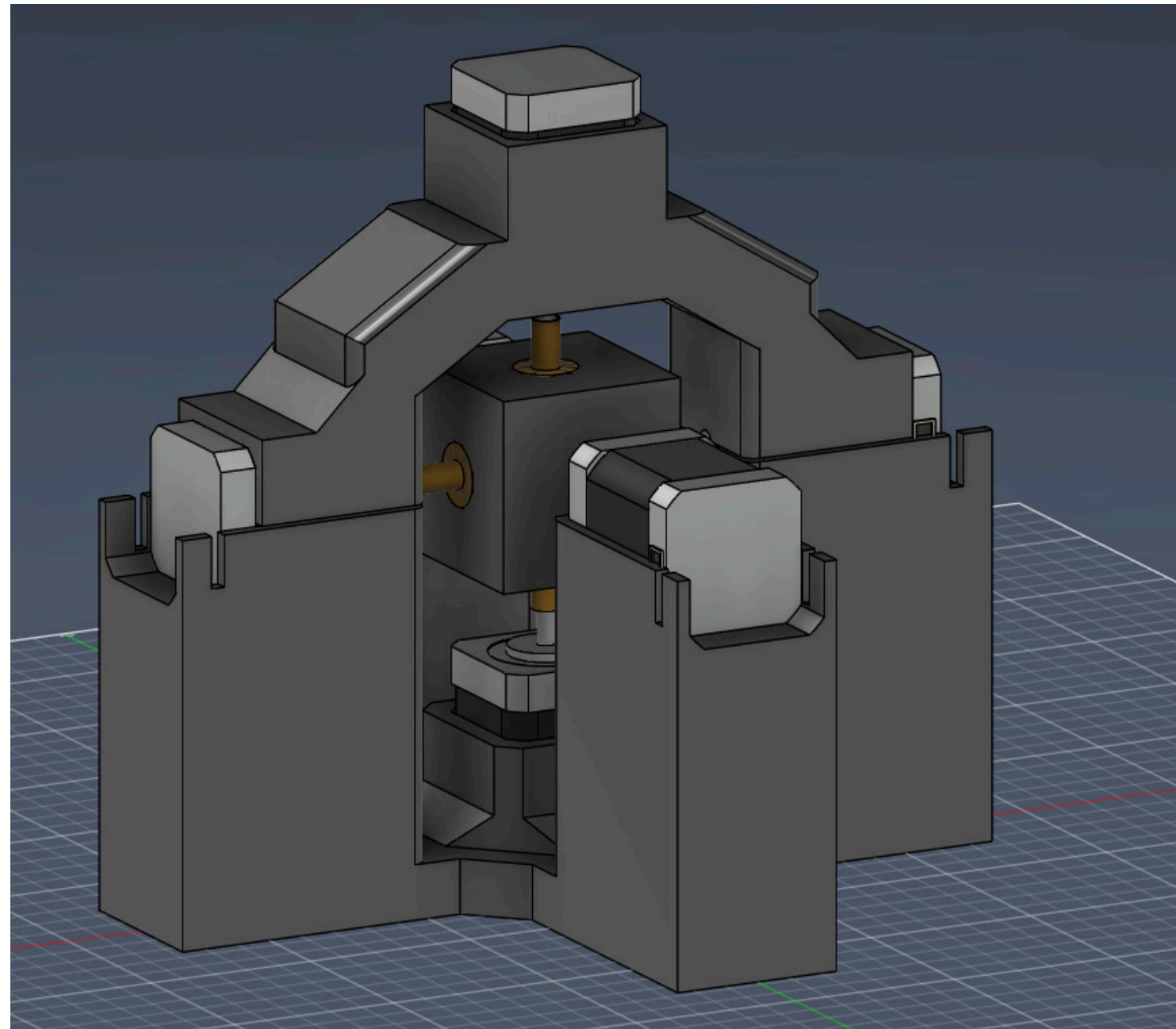
Engenharia do Sensor



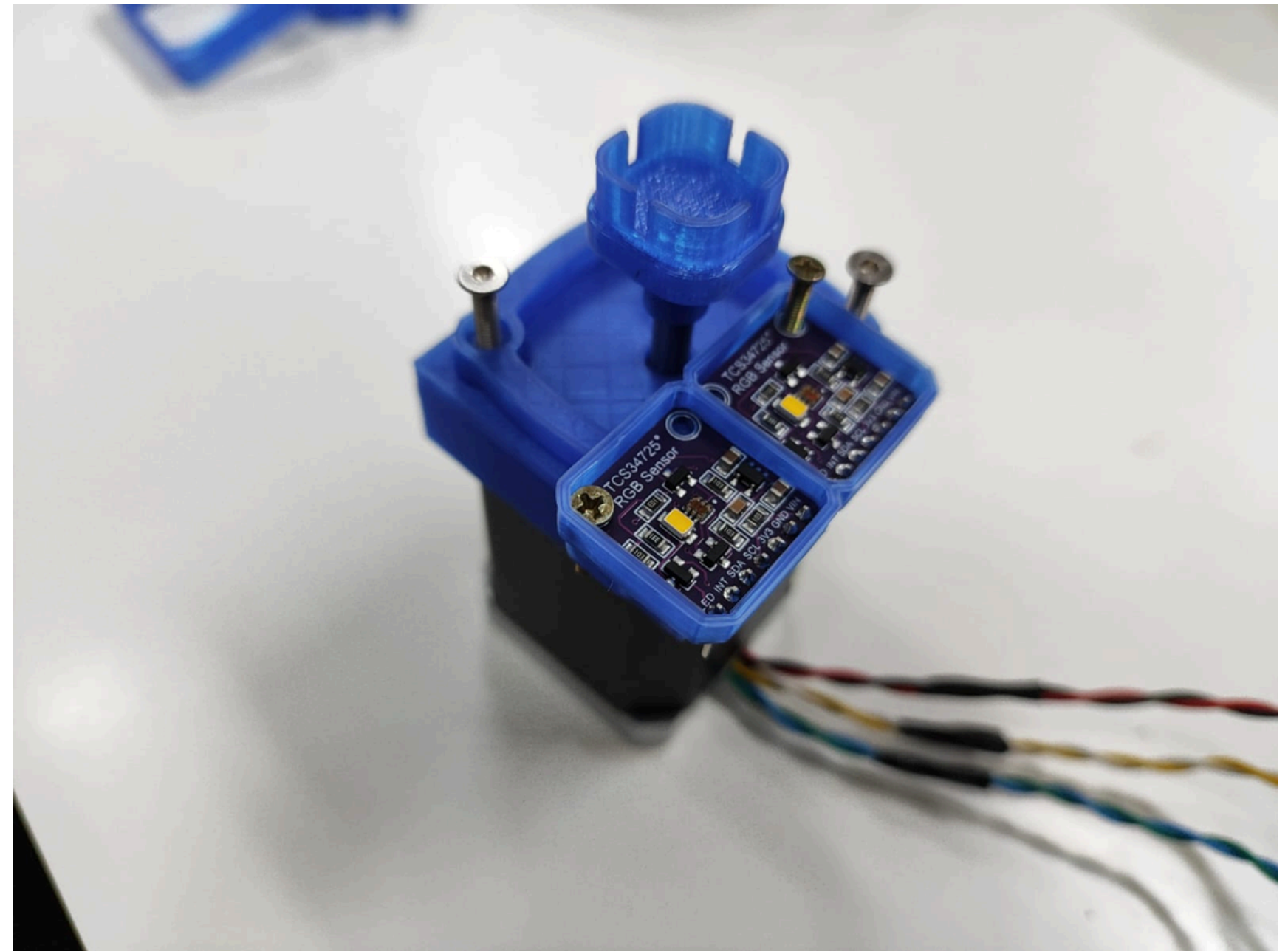
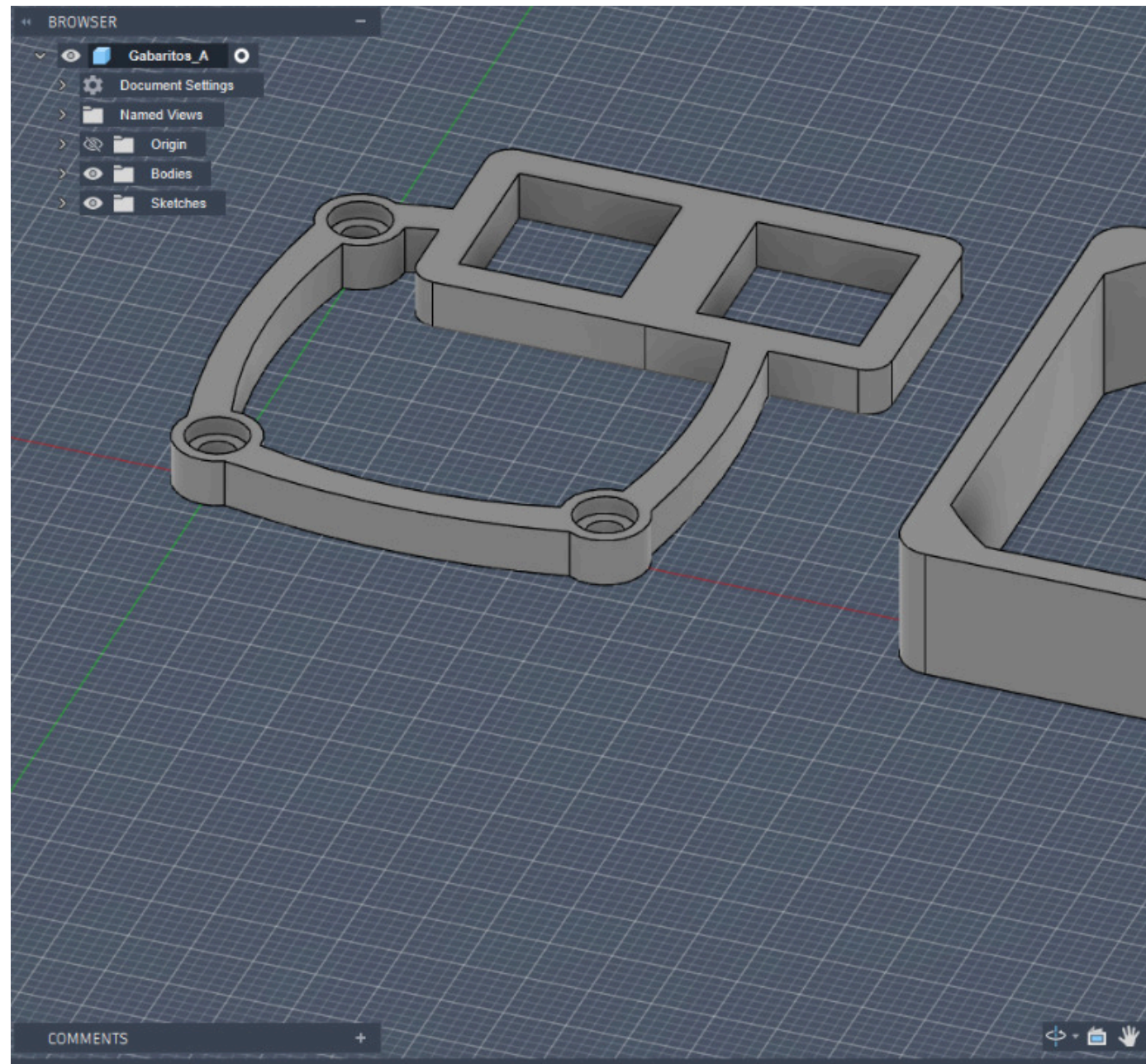
Exercício

Na Aula 3, construímos o sensor de cor (os "olhos").
Se temos um sensor que lê uma cor, qual é a solução de engenharia mais inteligente e barata para escanear todo o cubo mágico?

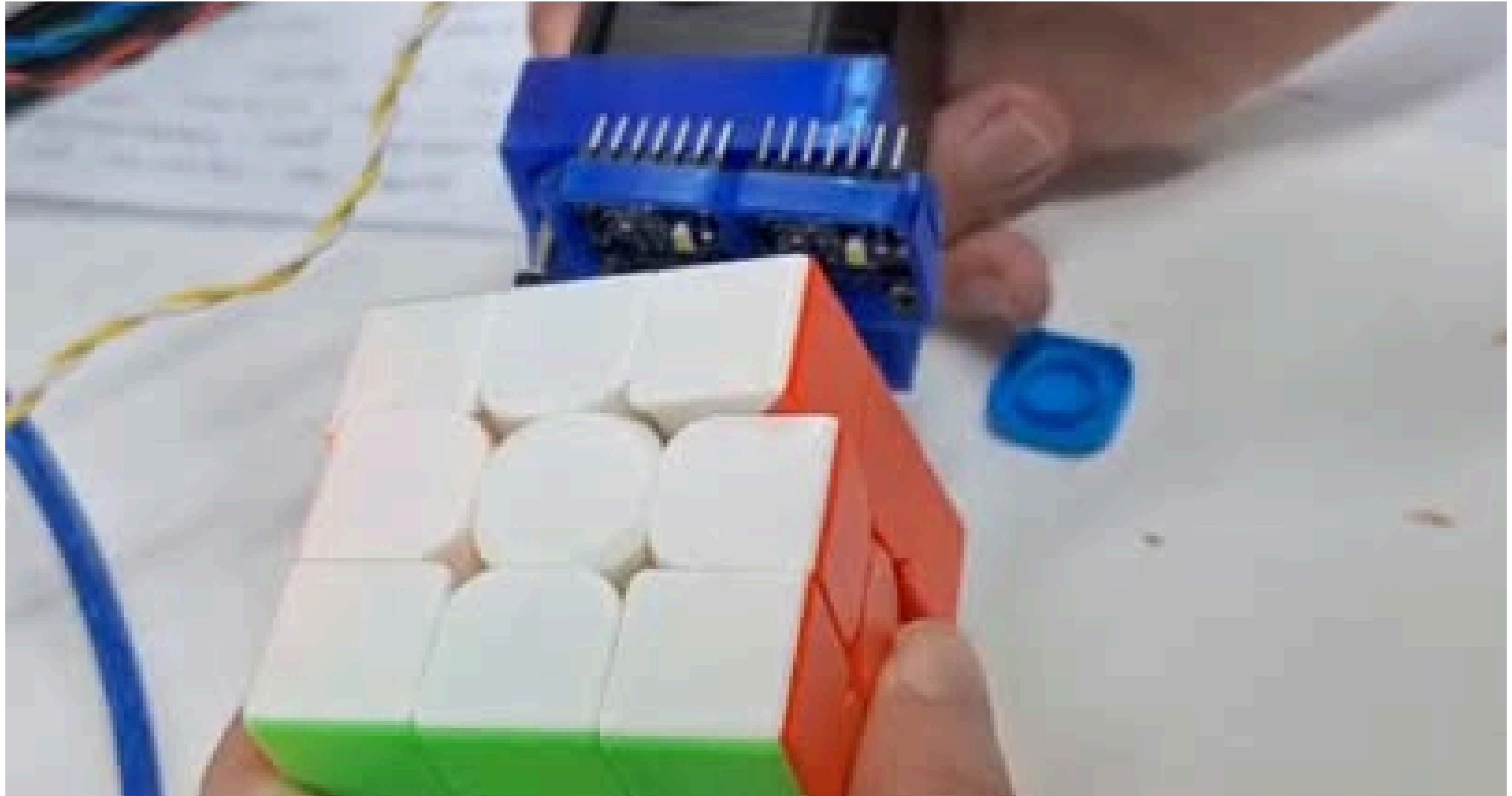
Solução



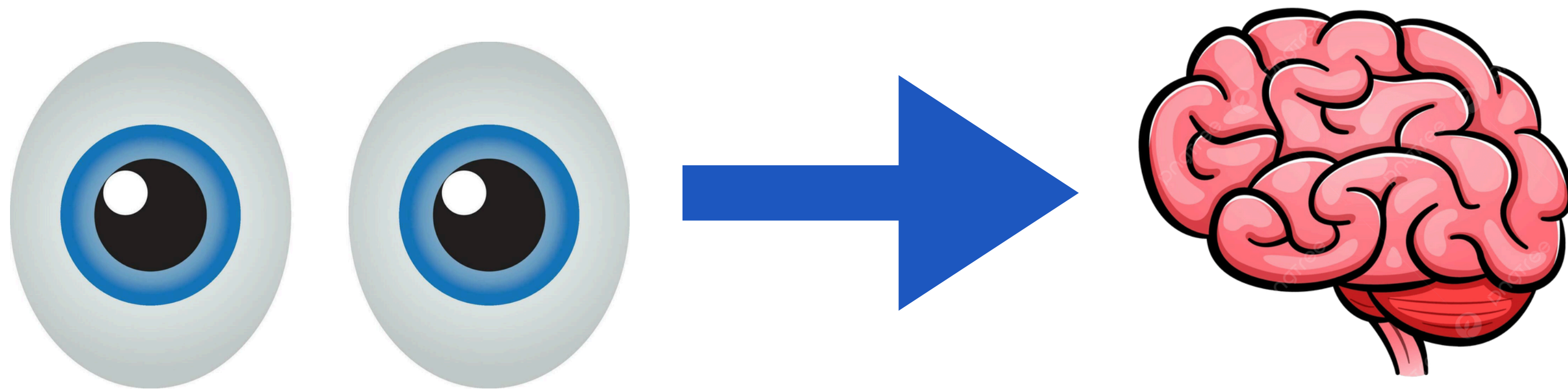
Solução

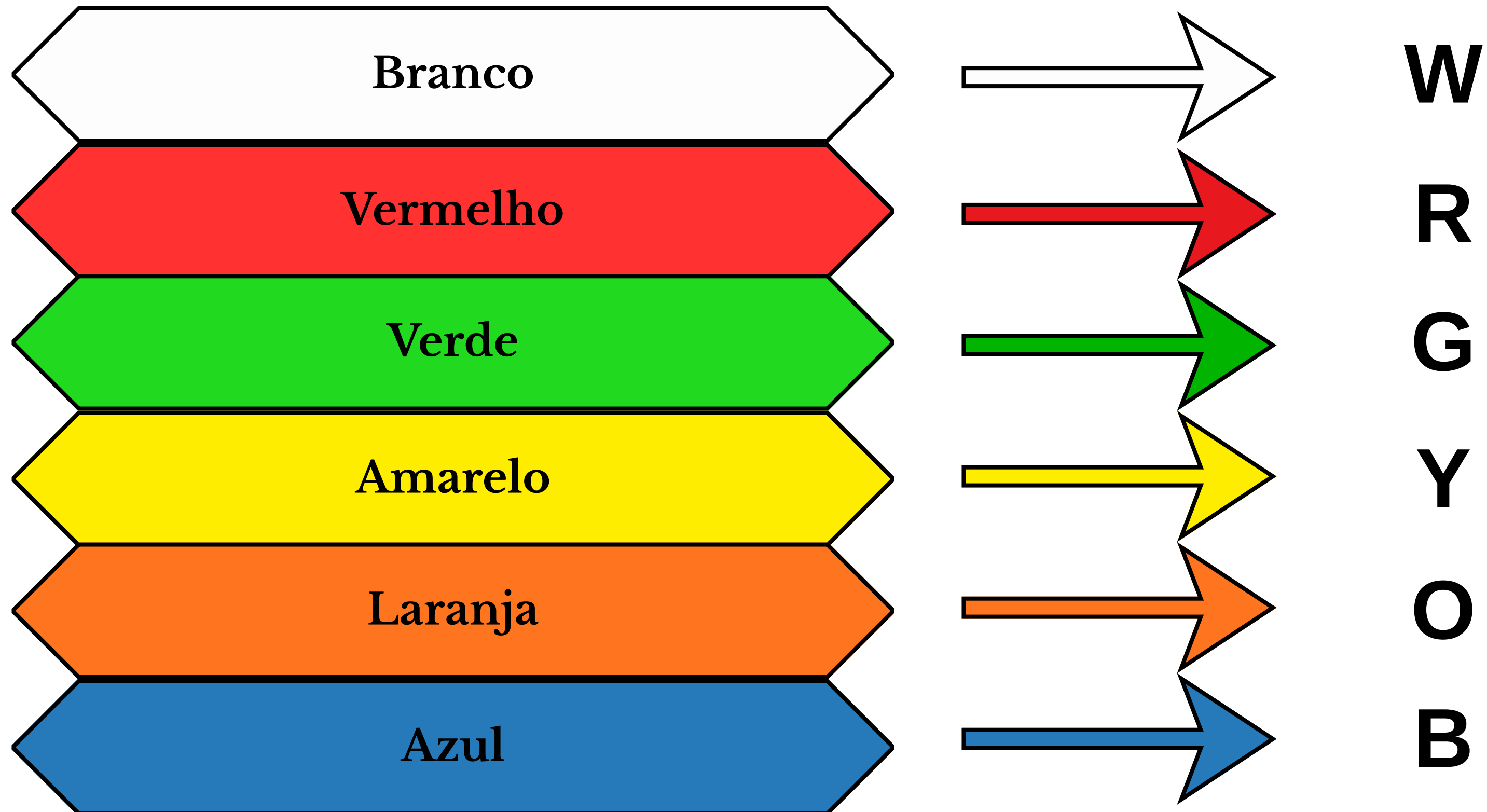


Solução



O Protocolo de Input (Olhos → Cérebro)





			45	46	47			
			44		48			
			43	42	41			
39	40	33	1	2	3	11	12	13
38		34	8		4	10		14
37	36	35	7	6	5	9	16	15
			17	18	19			
			24		20			
			23	22	21			
			25	26	27			
			32		28			
			31	30	29			

Up: [B,W,R,Y,W,G,R,R]

Right: [G,B,Y,Y,G,R,Y,W]

Front: [W,W,O,O,G,W,O,G]

Down: [B,R,O,B,Y,G,G,Y]

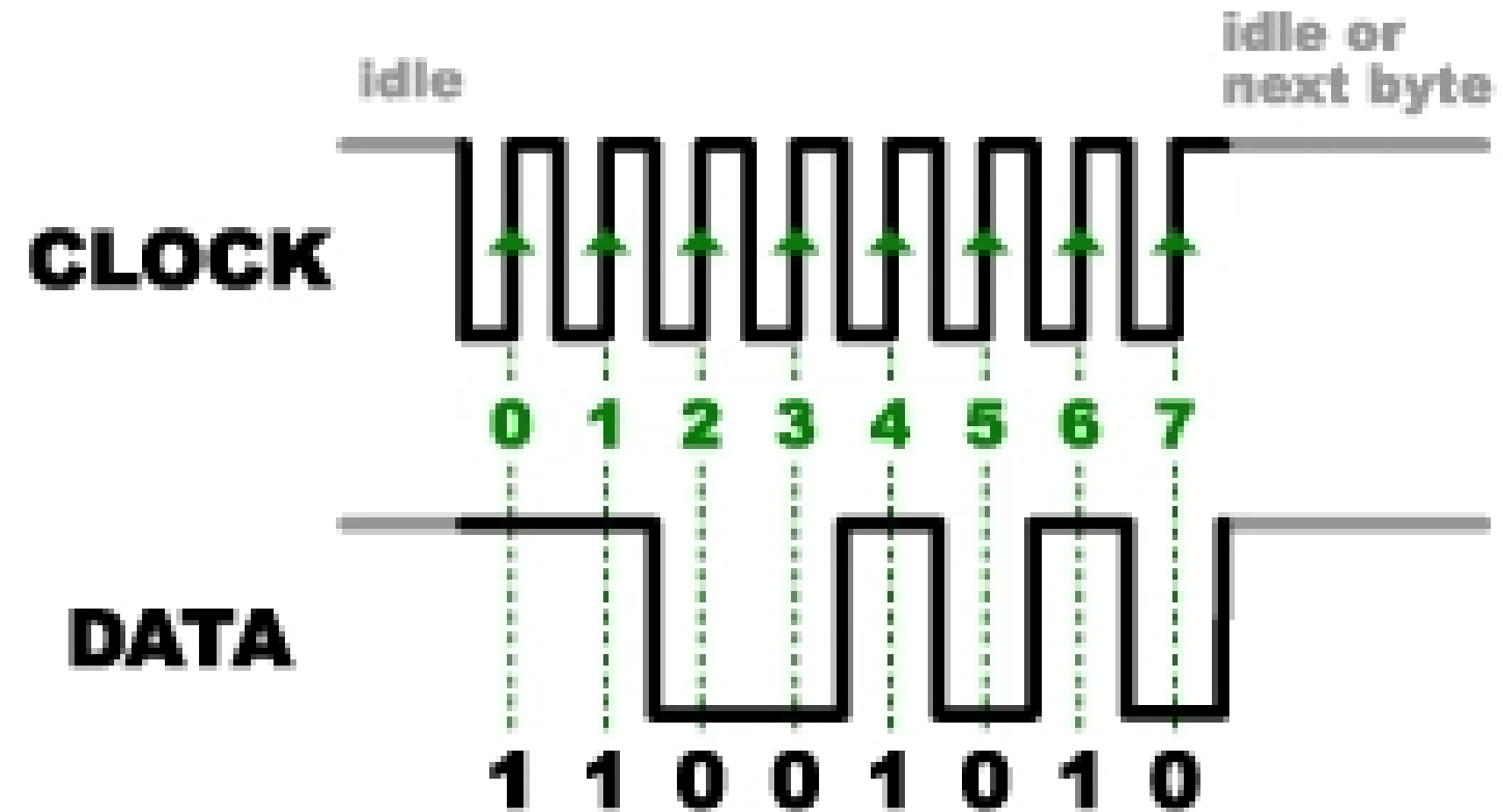
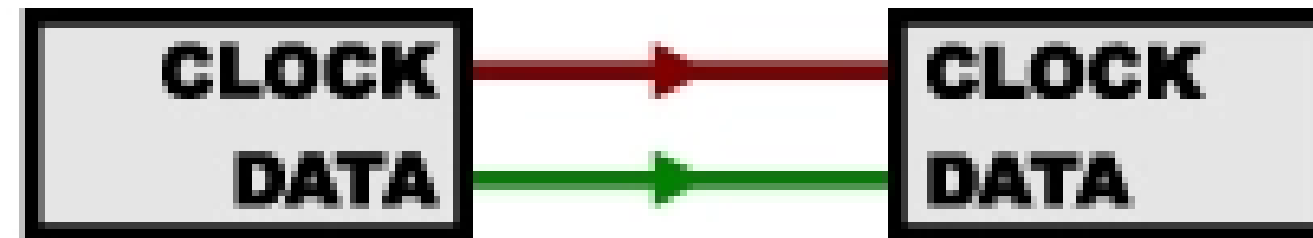
Left: [O,G,B,O,W,R,R,O]

Back: [B,B,Y,B,W,Y,R,O]

Exercício

- Embaralhe o cubo aleatoriamente.
- Utilizando a notação de cores (W, R, G, Y, O, B) e o protocolo de ordem que definimos, escreva a string completa que representa o estado atual de todas as 48 peças .

Comunicação Serial

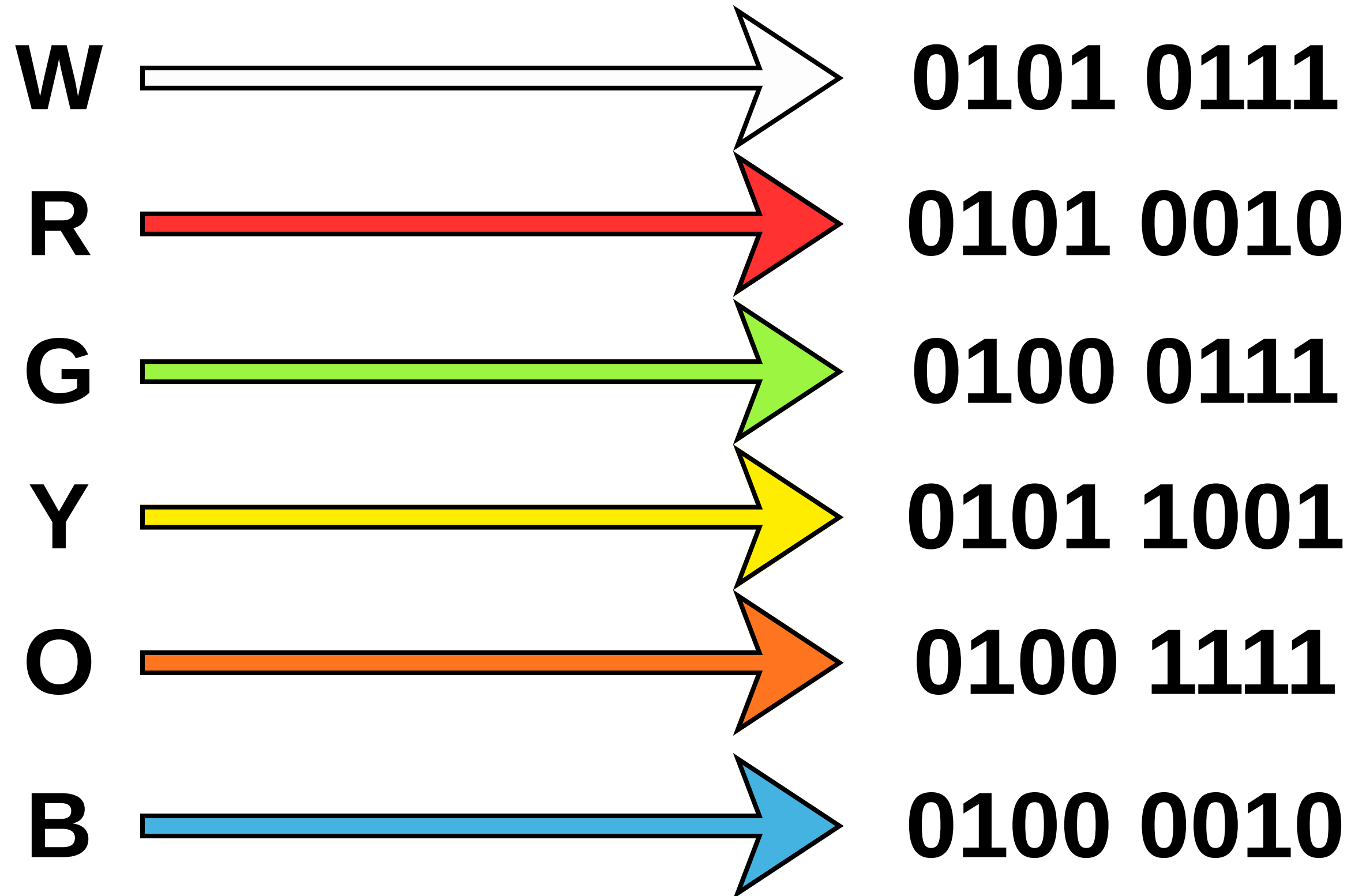


ASCII/UTF-8

Comunicação por Bytes

ASCII value	Char-acter	ASCII value	Char-acter	ASCII value	Char-acter	ASCII value	Char-acter	ASCII value	Char-acter	ASCII value	Char-acter
32	space	48	0	64	@	80	P	96	~	112	p
33	!	49	1	65	A	81	Q	97	a	113	q
34	"	50	2	66	B	82	R	98	b	114	r
35	#	51	3	67	C	83	S	99	c	115	s
36	\$	52	4	68	D	84	T	100	d	116	t
37	%	53	5	69	E	85	U	101	e	117	u
38	&	54	6	70	F	86	V	102	f	118	v
39	'	55	7	71	G	87	W	103	g	119	w
40	(	56	8	72	H	88	X	104	h	120	x
41	)	57	9	73	I	89	Y	105	i	121	y
42	*	58	:	74	J	90	Z	106	j	122	z
43	+	59	;	75	K	91	[	107	k	123	{
44	,	60	<	76	L	92	\	108	l	124	
45	-	61	=	77	M	93	]	109	m	125	}
46	.	62	>	78	N	94	^	110	n	126	~
47	/	63	?	79	O	95	_	111	o	127	DEL

Exemplo

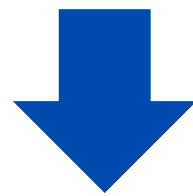
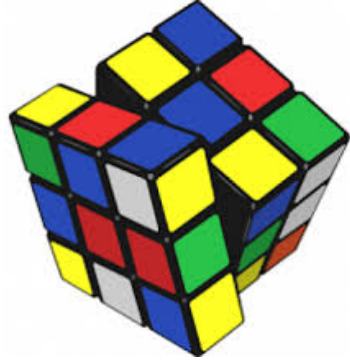


Exercício

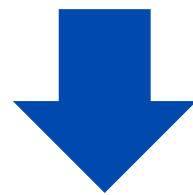
Converta o início da string de 48 caracteres que representa o estado do cubo para a sua representação binária (em bytes), conforme o protocolo de codificação serial (ASCII/UTF-8). Mostre a sequência de 0s e 1s que o Python lerá.

Resumo

Arduíno



[B,W,R,Y,...,W,G,R,R]



[1000101...010110]

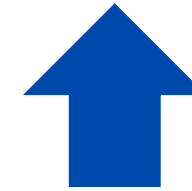
Baud Rate
Start Bit
Stop Bit
Clock

0100001110...

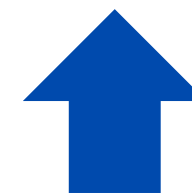
Serial

Python

Calcula a Solução



[B,W,R,Y,...,W,G,R,R]



[1000101...010110]

Métodos de Solução do Cubo Mágico

Camadas

CFOP

Roux

Eka

ZZ

Petrus

3-Style

Heise

OP/OP

M2/OP

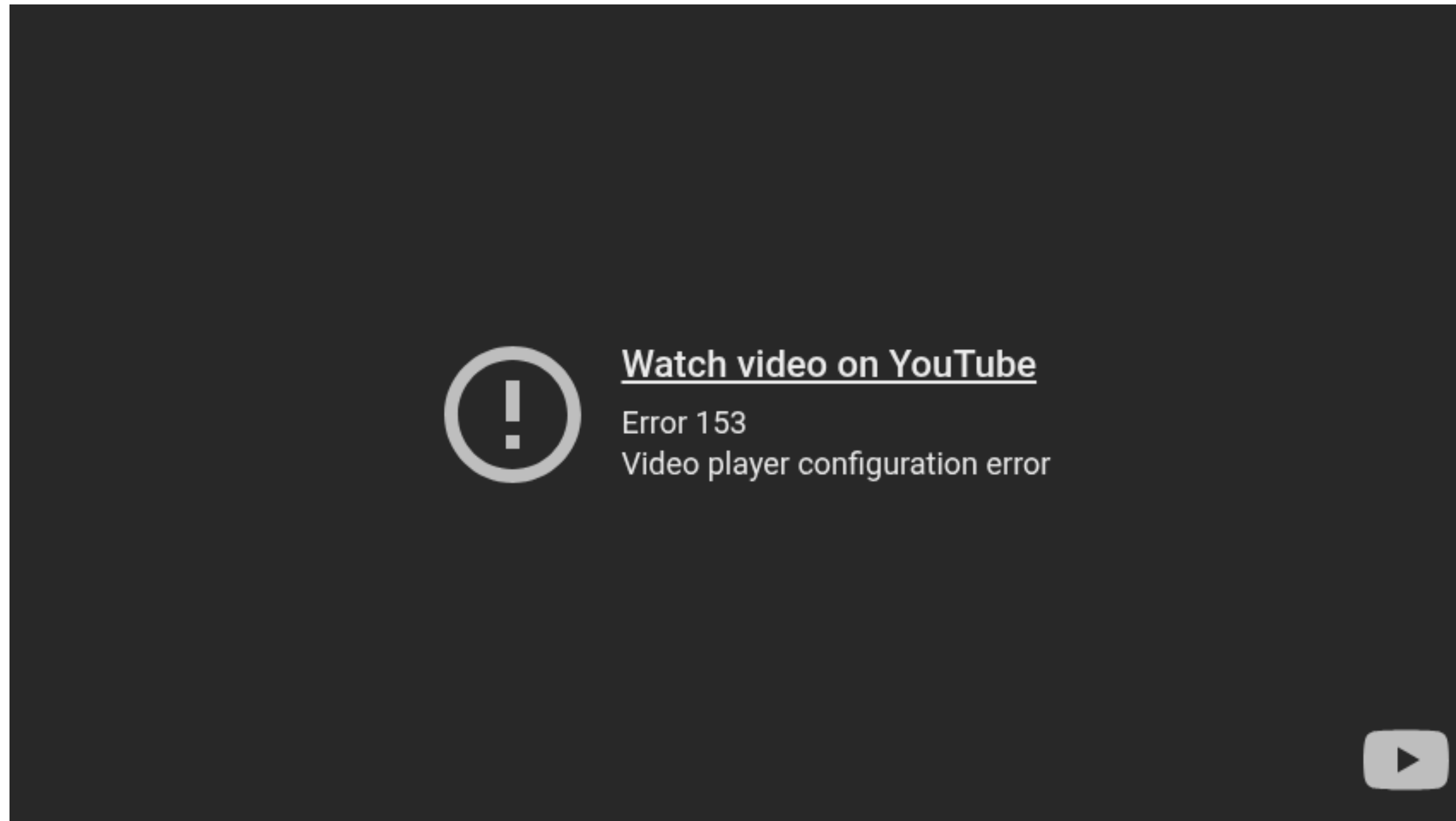
ZBLL

Keyhole

DR

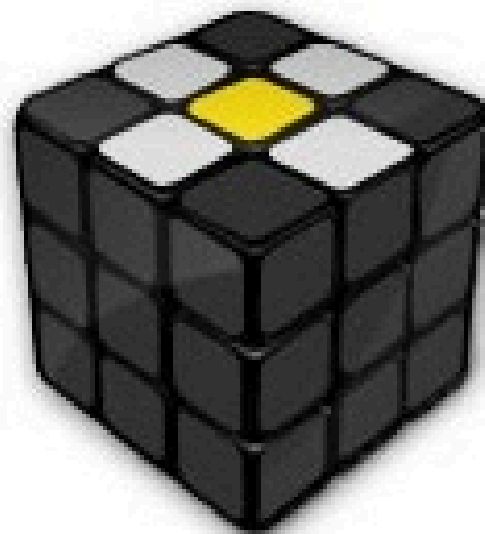
Orozco

Método Básico de Camadas

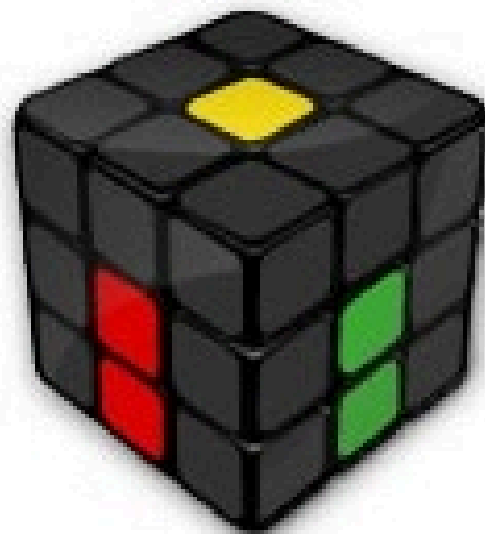


3x3x3 Básico

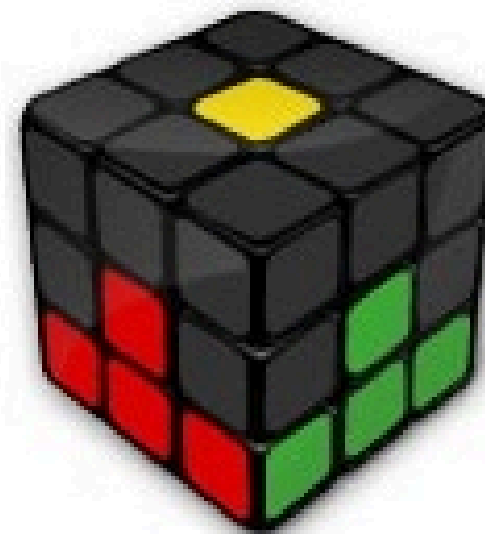
1º Passo
Cruz no amarelo



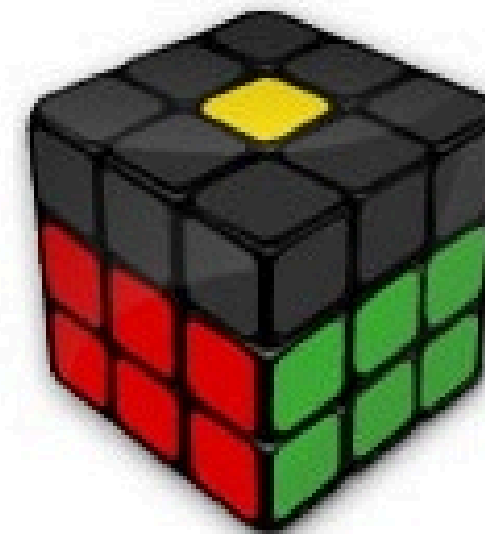
2º Passo
Permutação da cruz



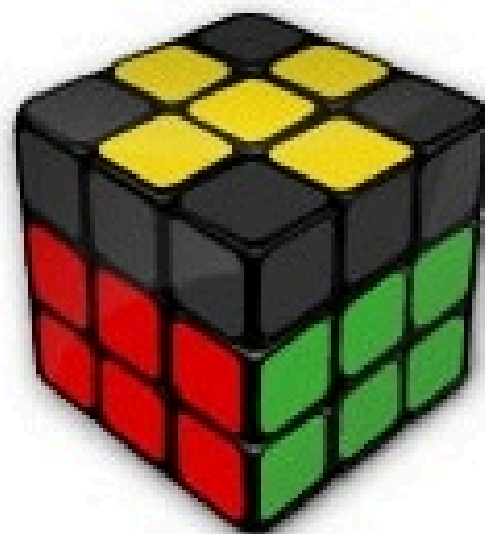
3º Passo
Camada branca



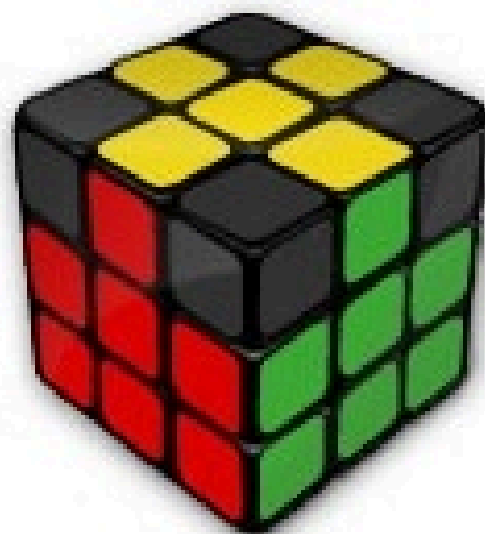
4º Passo
Camada do meio



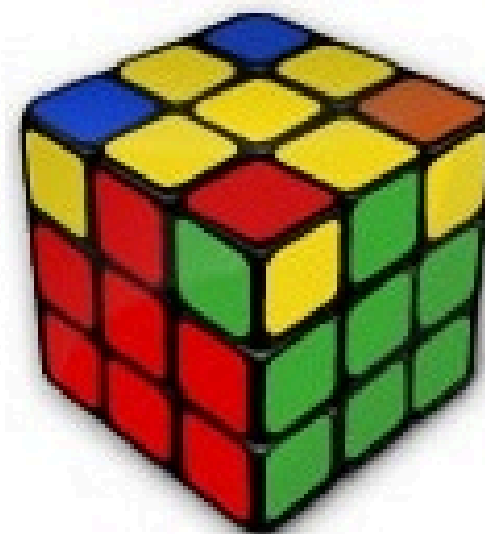
5º Passo
Orientação da
cruz amarela



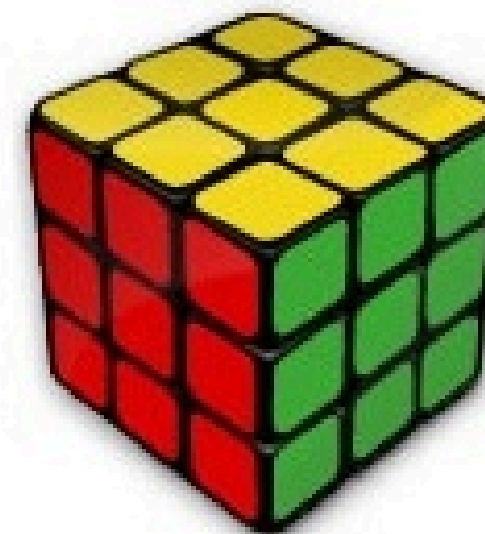
6º Passo
Permutação da
cruz amarela



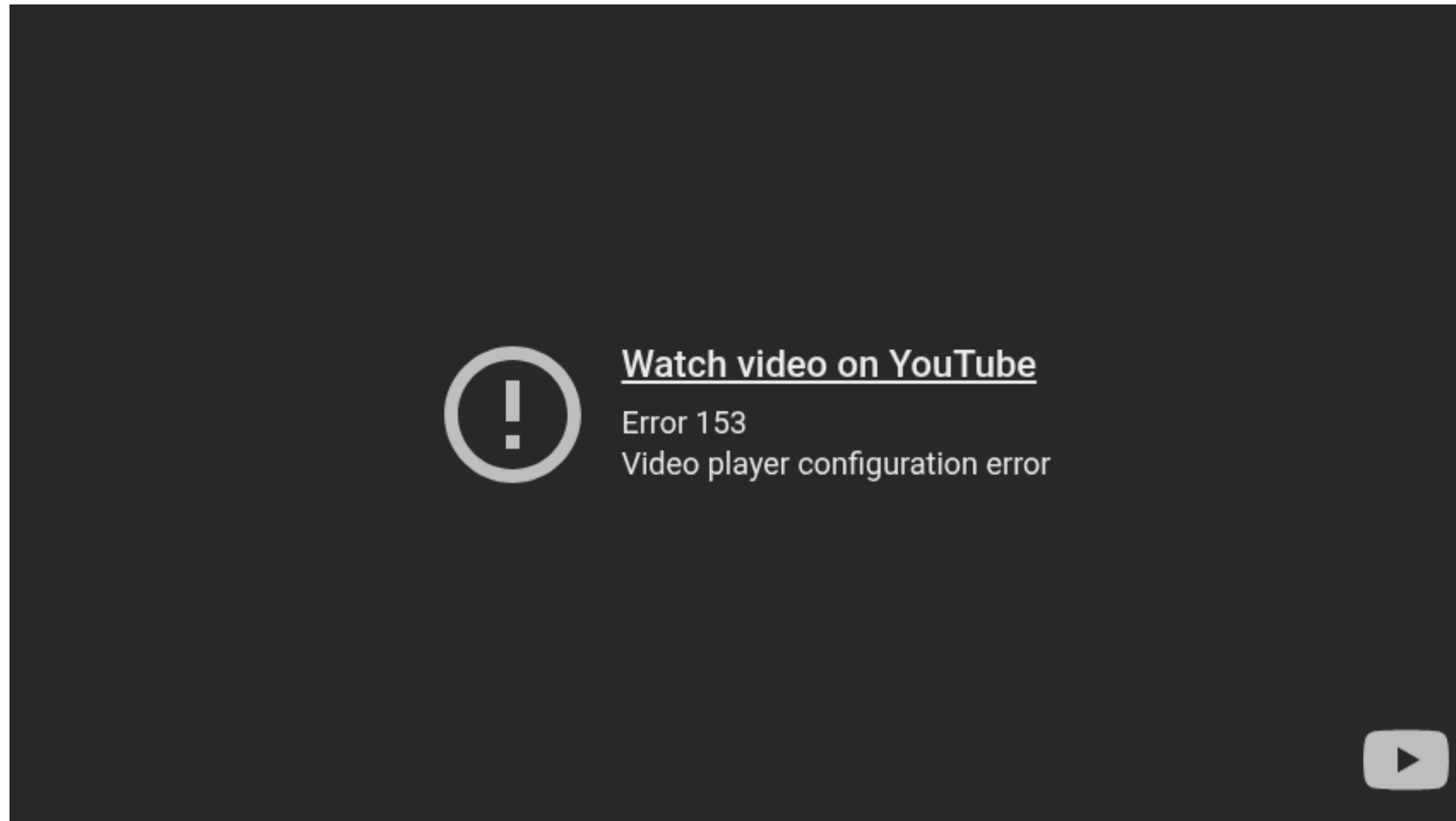
7º Passo
Permutação das
quinas amarelas

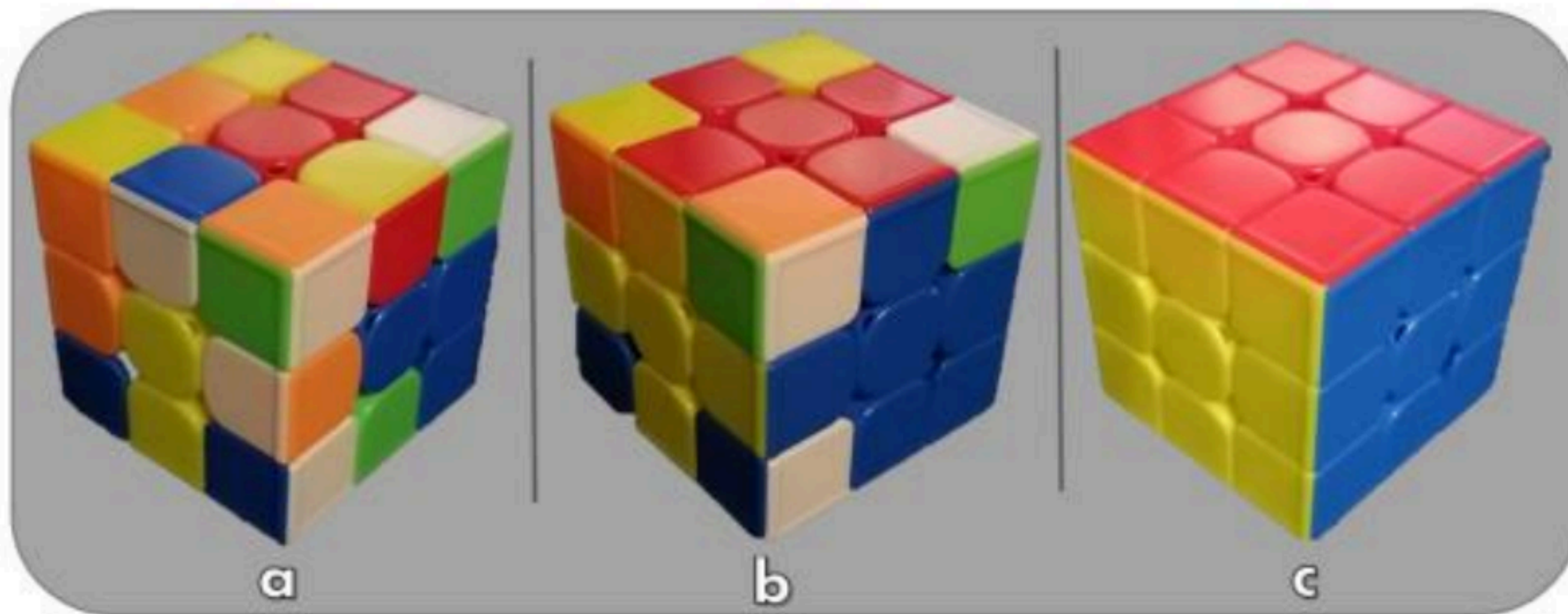


8º Passo
Organização das
quinas

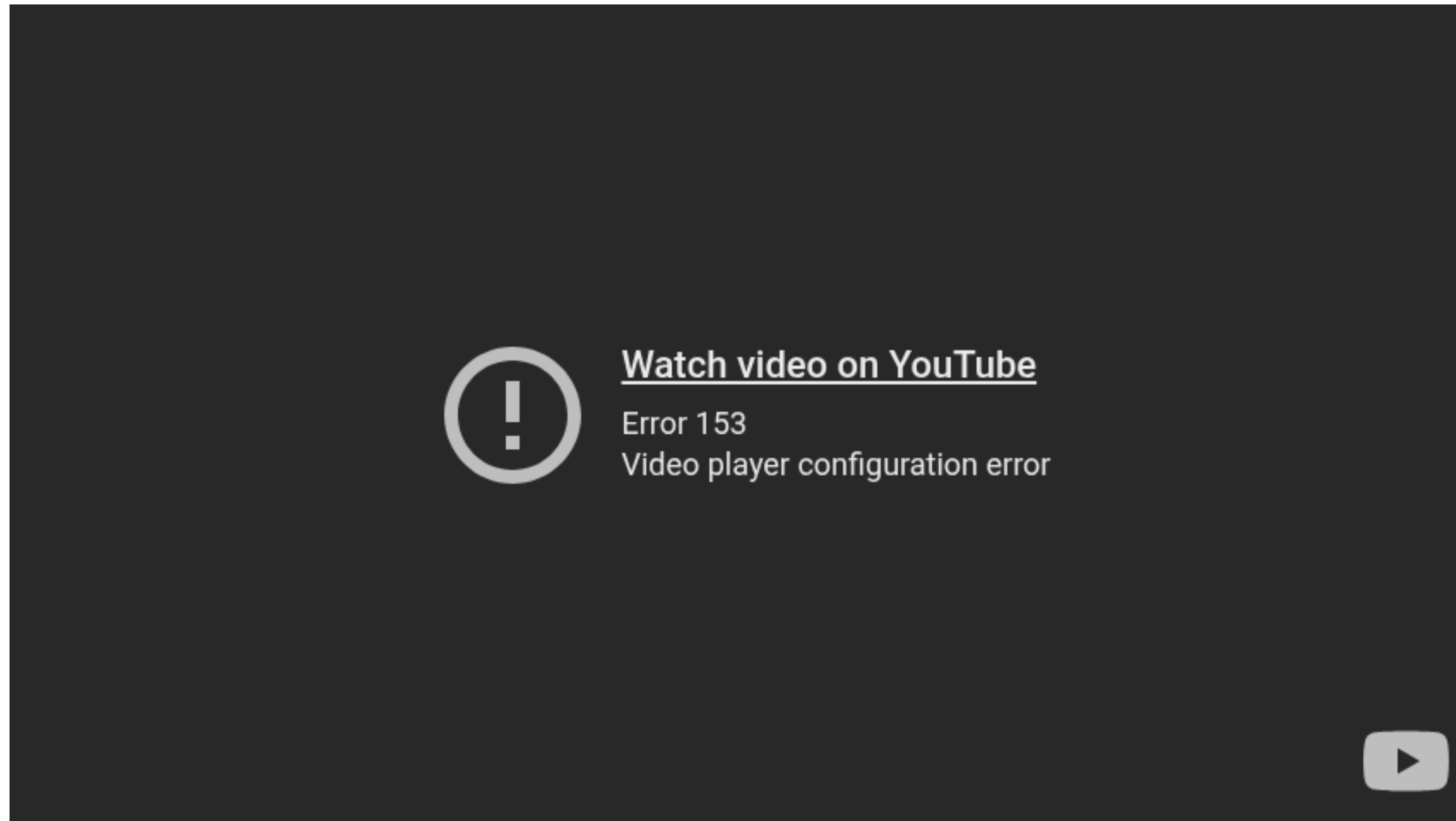


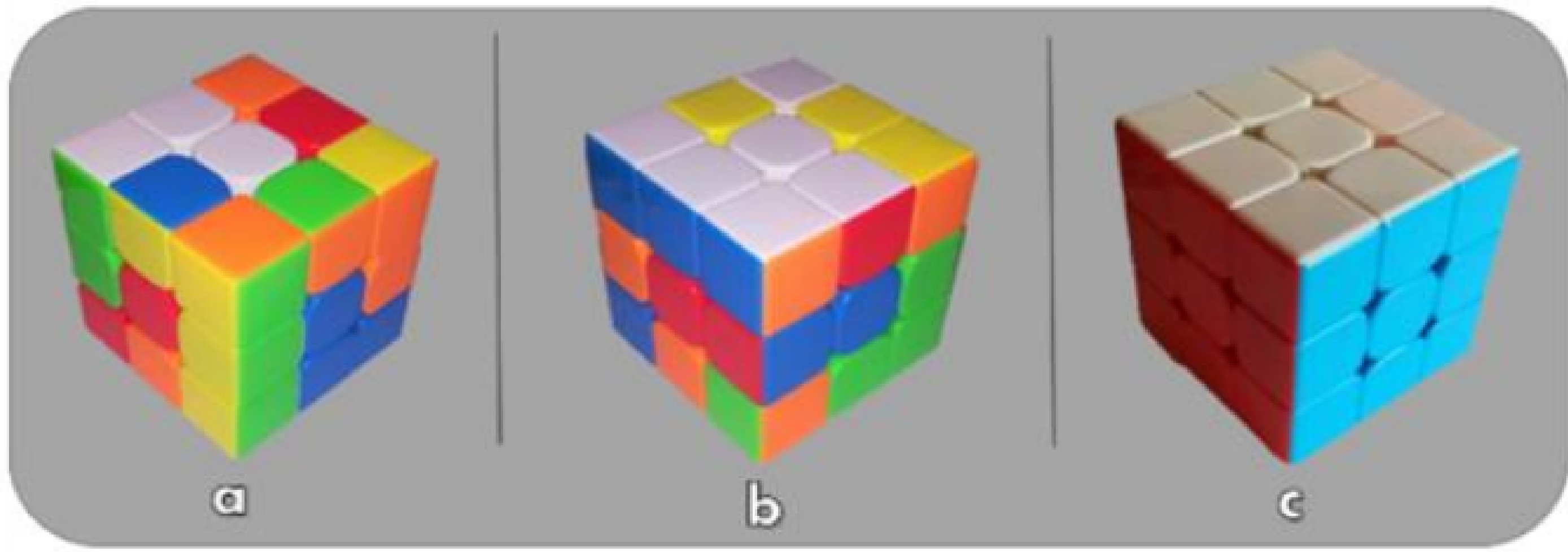
Método Vendado M2/OP



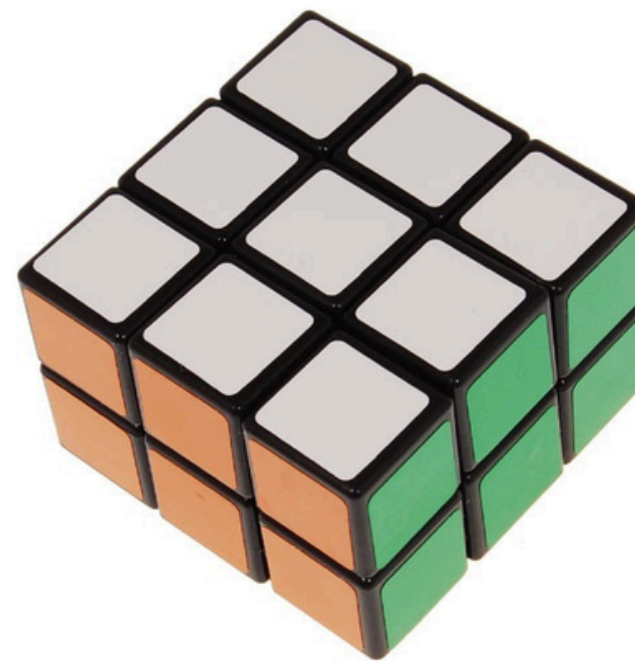


Método Kociemba "Número de Deus"





Domino's Reduction
 $\{U, D, R2, L2, F2, B\}$



Herbert Kociemba, matemático alemão, desenvolveu seu método nos anos 1990.

Fase 1 (Max. 12 movimentos): Levar o cubo para um subconjunto de estados mais simples (o "Domino's Reduction")

Fase 2 (Max. 18 movimentos): Resolver o cubo a partir desse estado simplificado.

O máximo de movimentos garantidos por este método para resolver qualquer estado é 30 movimentos.

O Cubo Mágico tem aproximadamente 43 quintilhões de embaralhamentos o que torna impossível testar todas as soluções.

O método de Kociemba permitiu uma generalização: precisaram resolver apenas um subconjunto específico e representativo de todos os embaralhamentos.

Em 2010, usando o poder de processamento da Google (milhares de horas de CPU), eles testaram por exaustão todos os casos necessários.

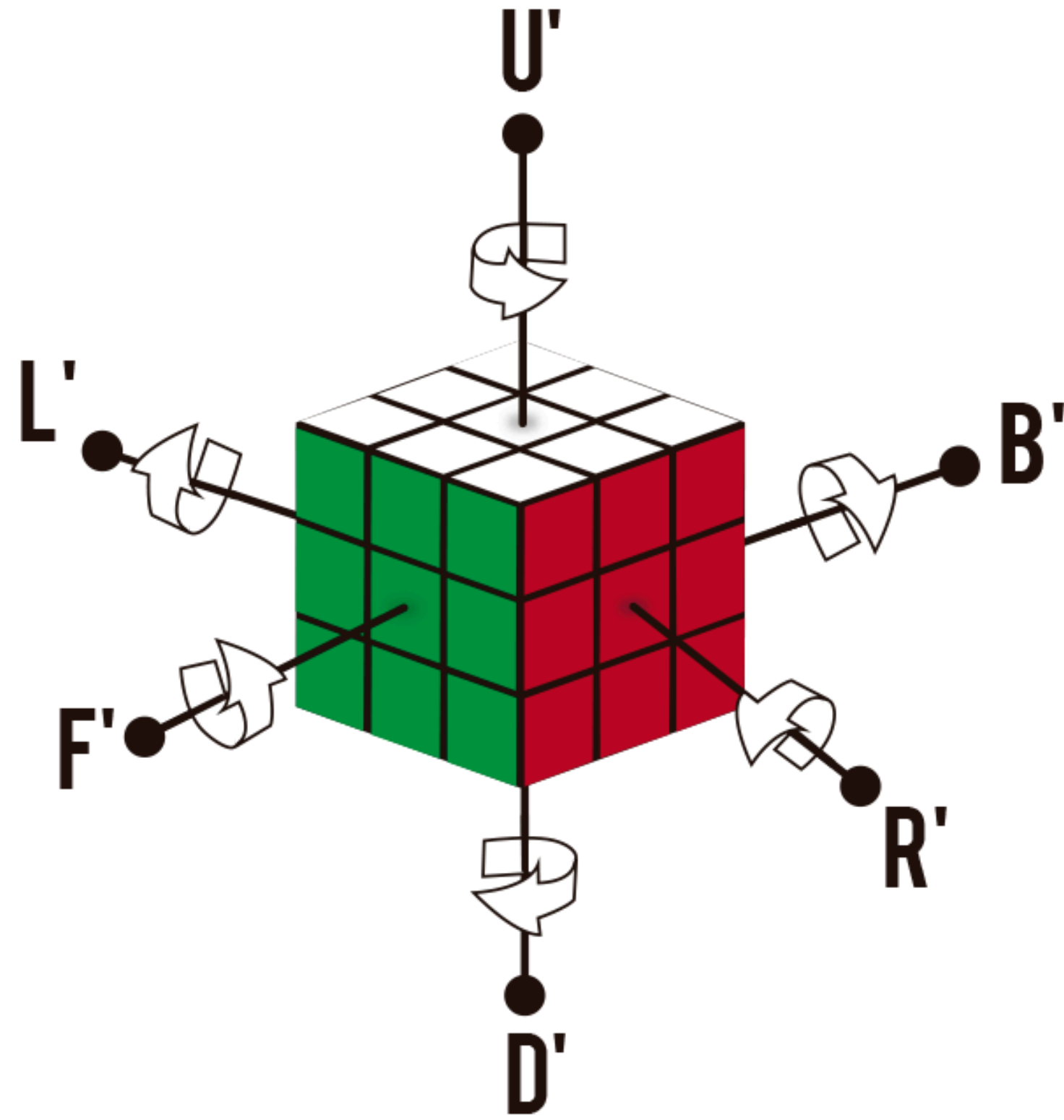
Conclusão Final: O número máximo de movimentos mínimos necessários para resolver qualquer estado do Cubo Mágico é 20.

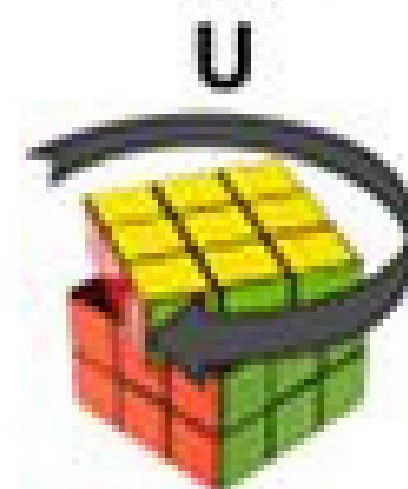
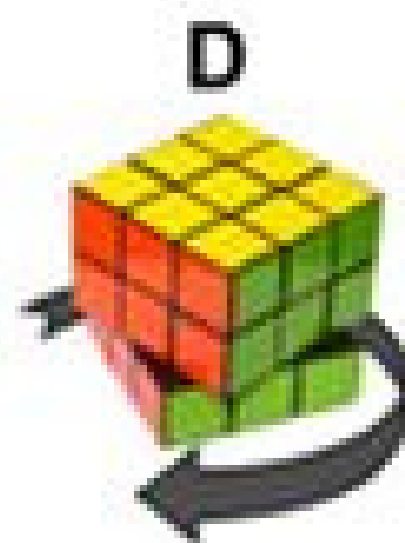
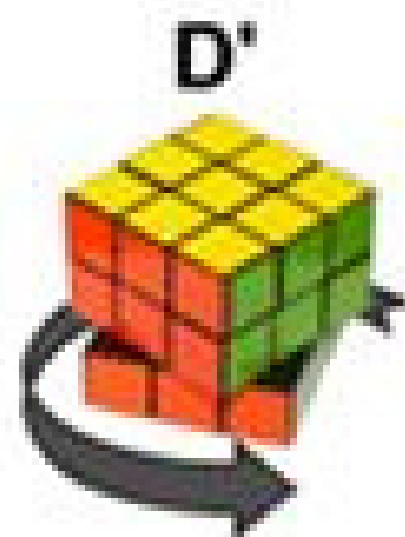
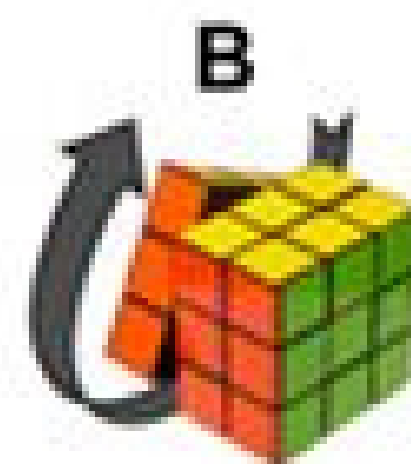
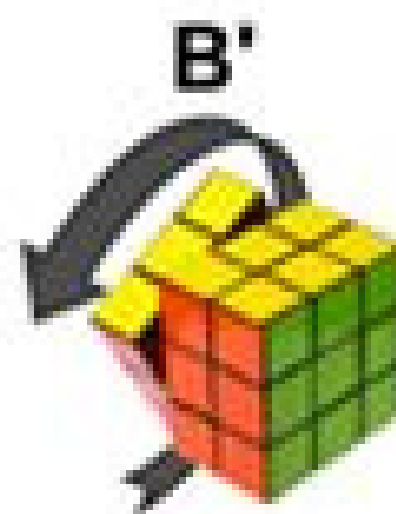
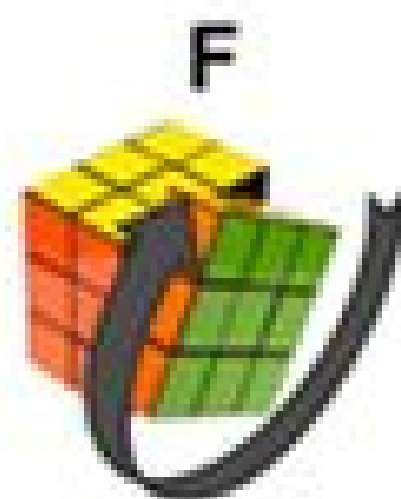
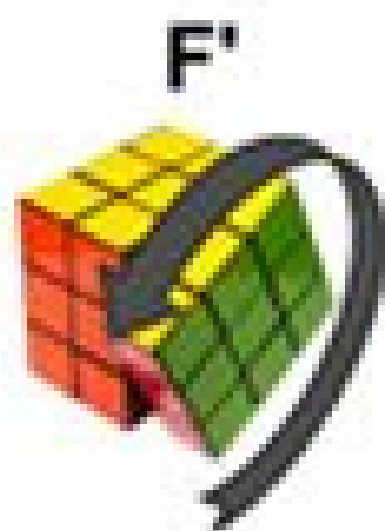
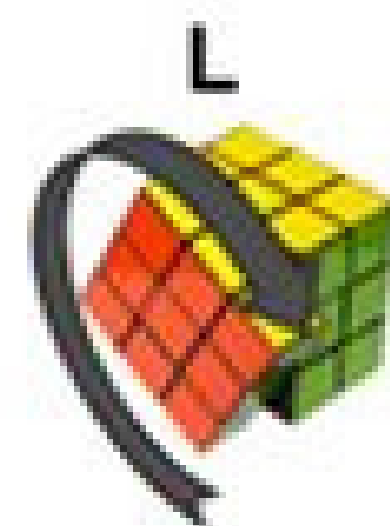
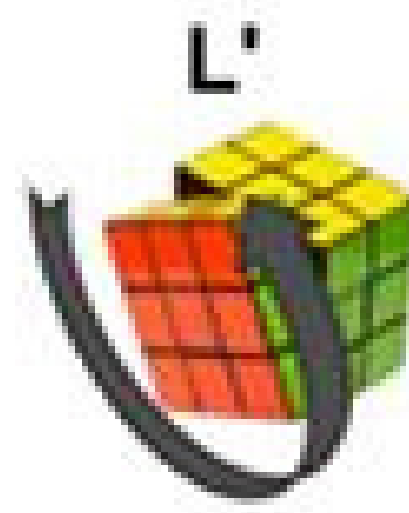
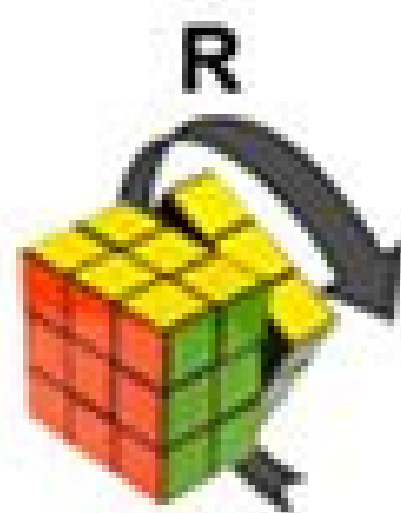
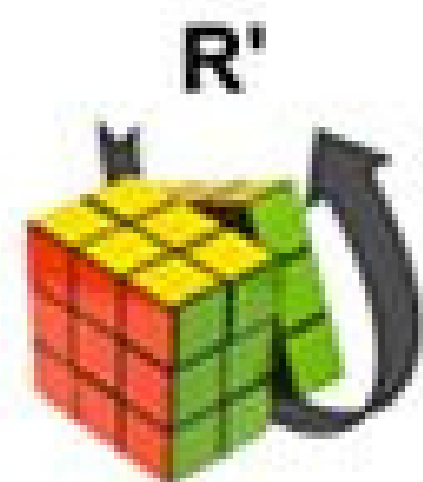
Qual método nosso robô irá usar?

O nosso robô irá utilizar o princípio dos Comutadores (método vendado) para resolver o Cubo Mágico.

Porque?

Notação Oficial dos movimentos do Cubo Mágico





Exercício

Com o cubo mágico resolvido tente executar a seguinte sequencia de movimentos:

$R U R' F' R U R' U' R' F R^2 U' R' U'$

Lógica de Comutadores

Ideia Matemática

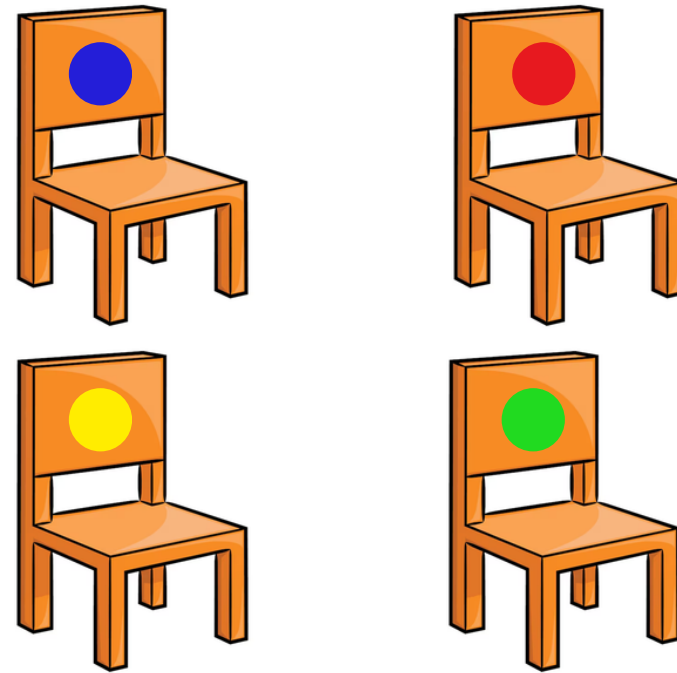
$$[A \ B \ A' \ B']$$

Com A e B uma sequencia de movimentos, onde A' é o inverso de A e B' é o inverso de B

Exemplo

Estado Inicial

Sala 1

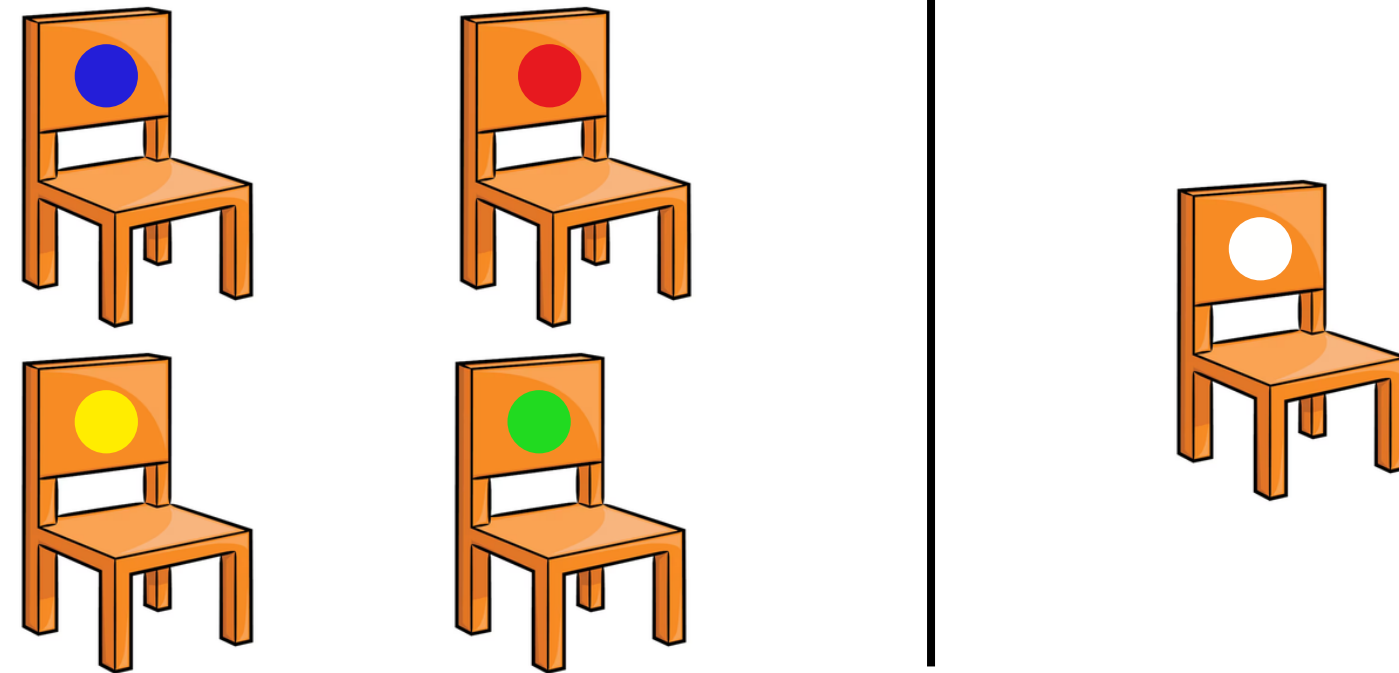


Sala 2

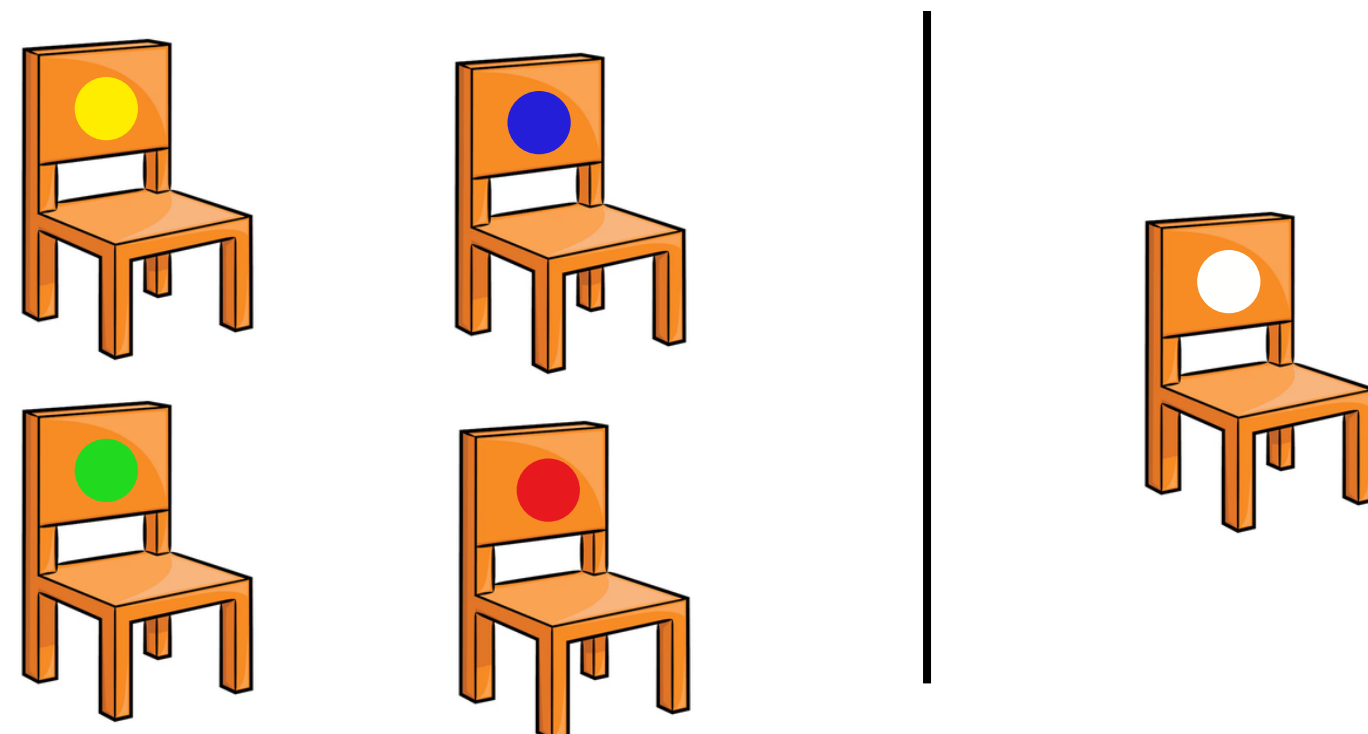
Objetivo:

Comutar as cadeiras Vermelho, Verde, Branco sem mudar as cadeiras Azul e Amarelo

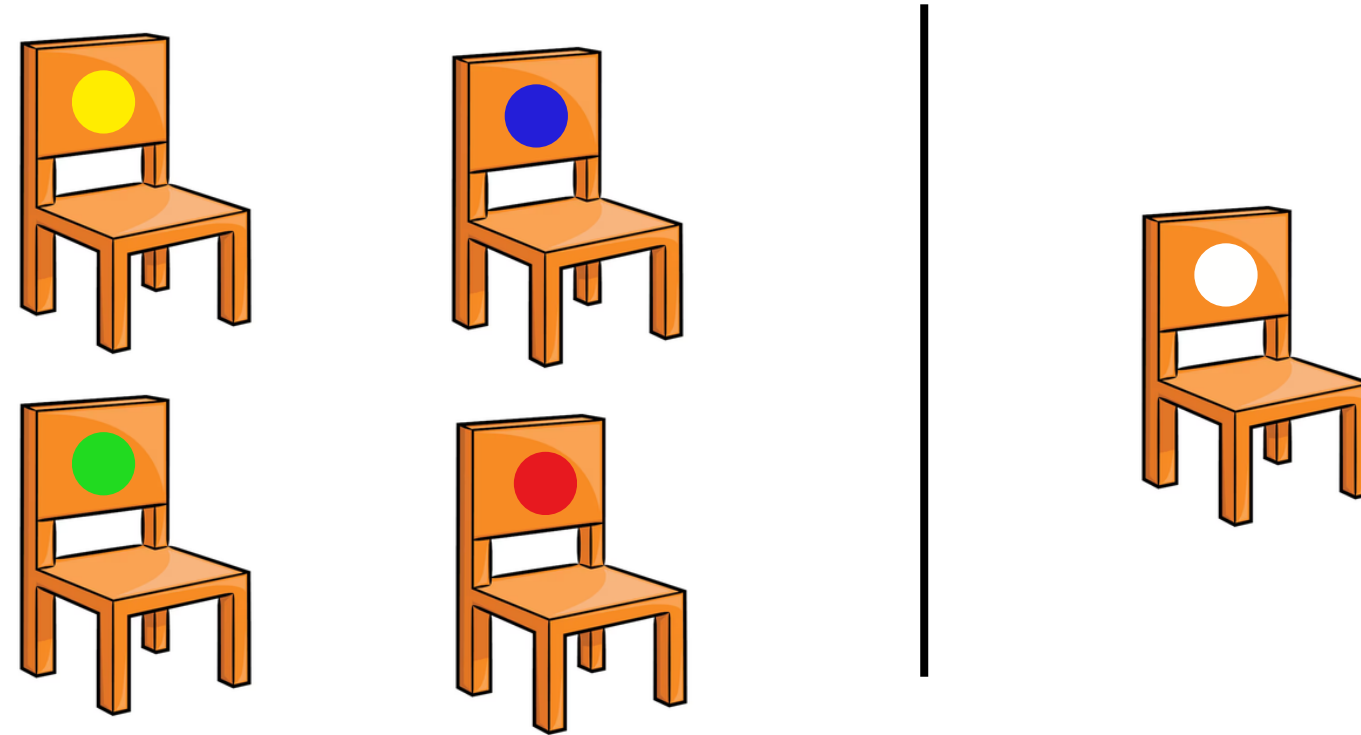
Exemplo



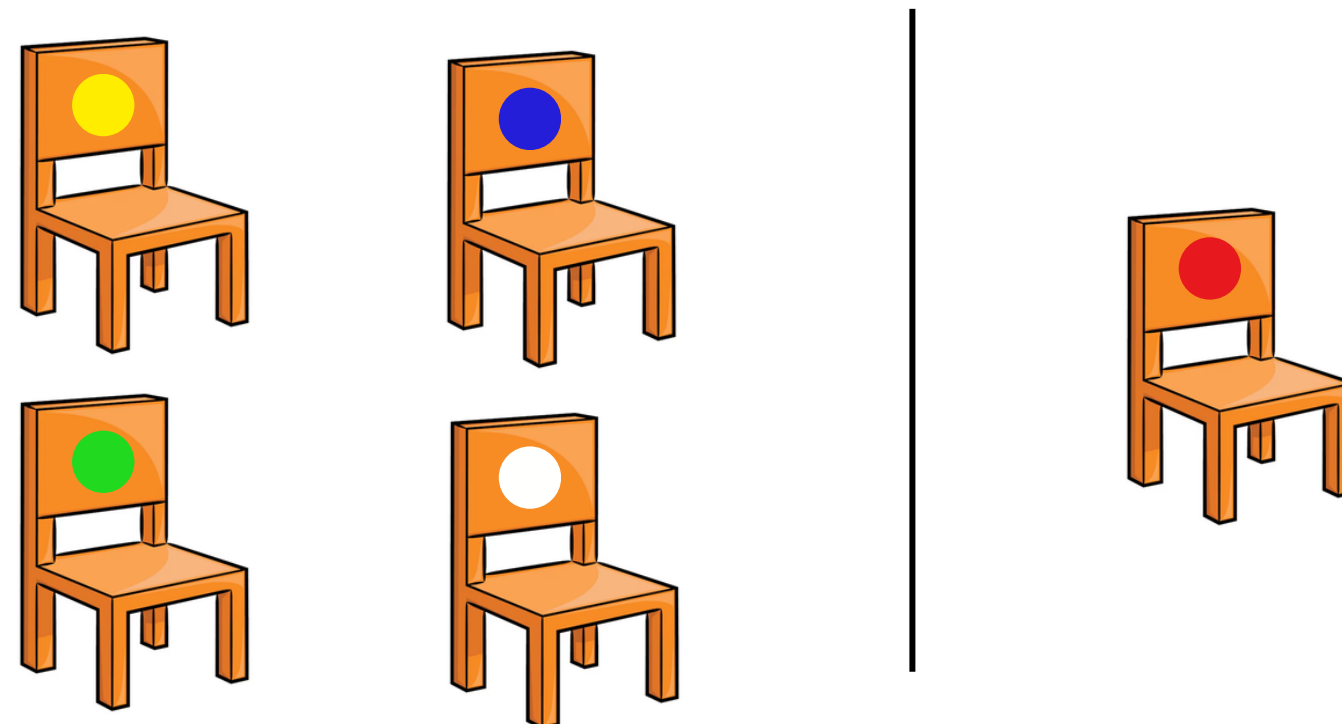
A: rotaciona as cadeiras da sala 1 no sentido horário



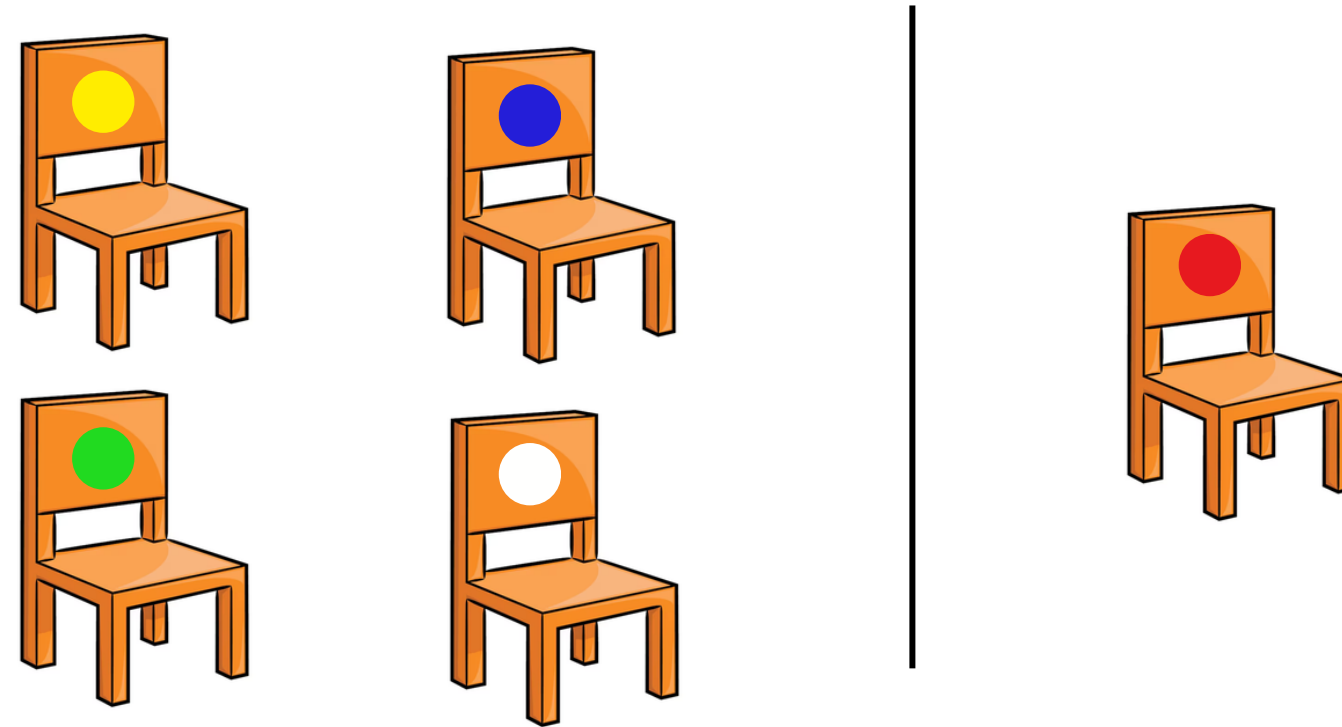
Exemplo



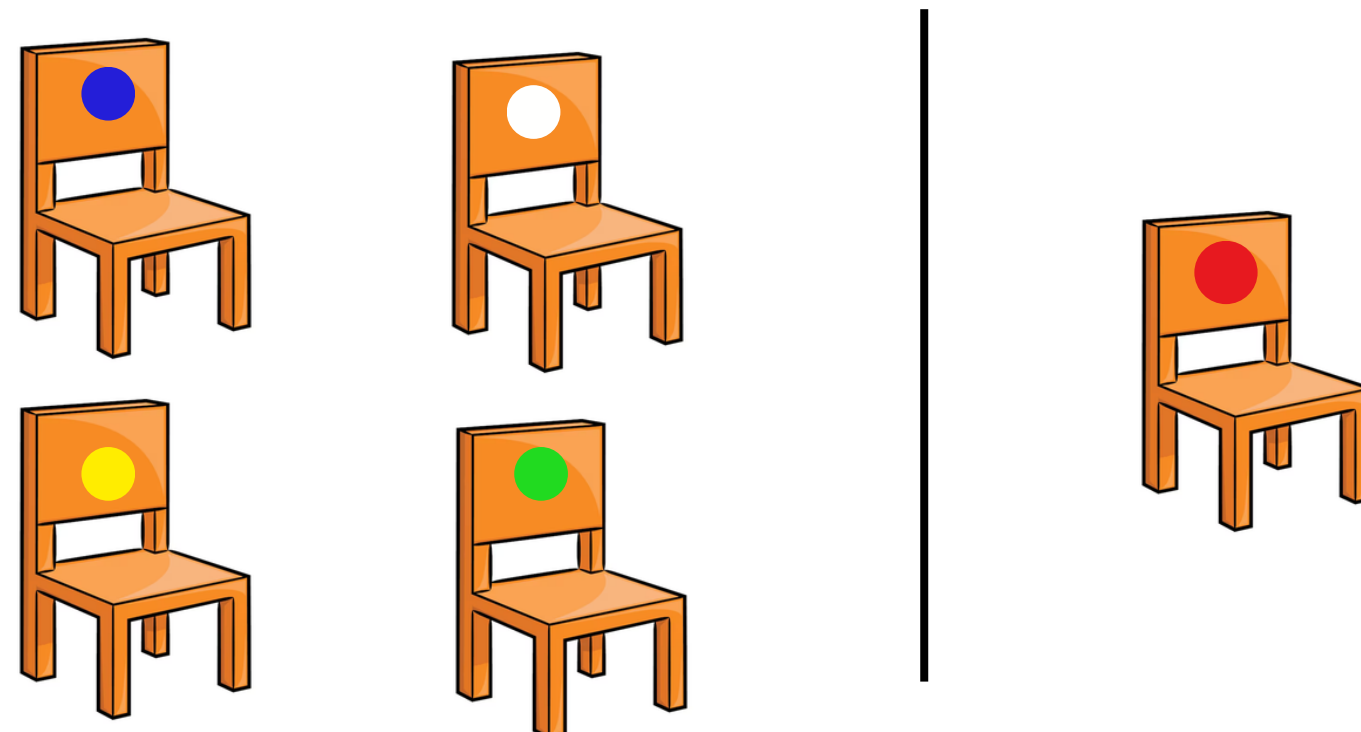
B: troca a cadeira da sala 2 com a cadeira do canto da sala 1



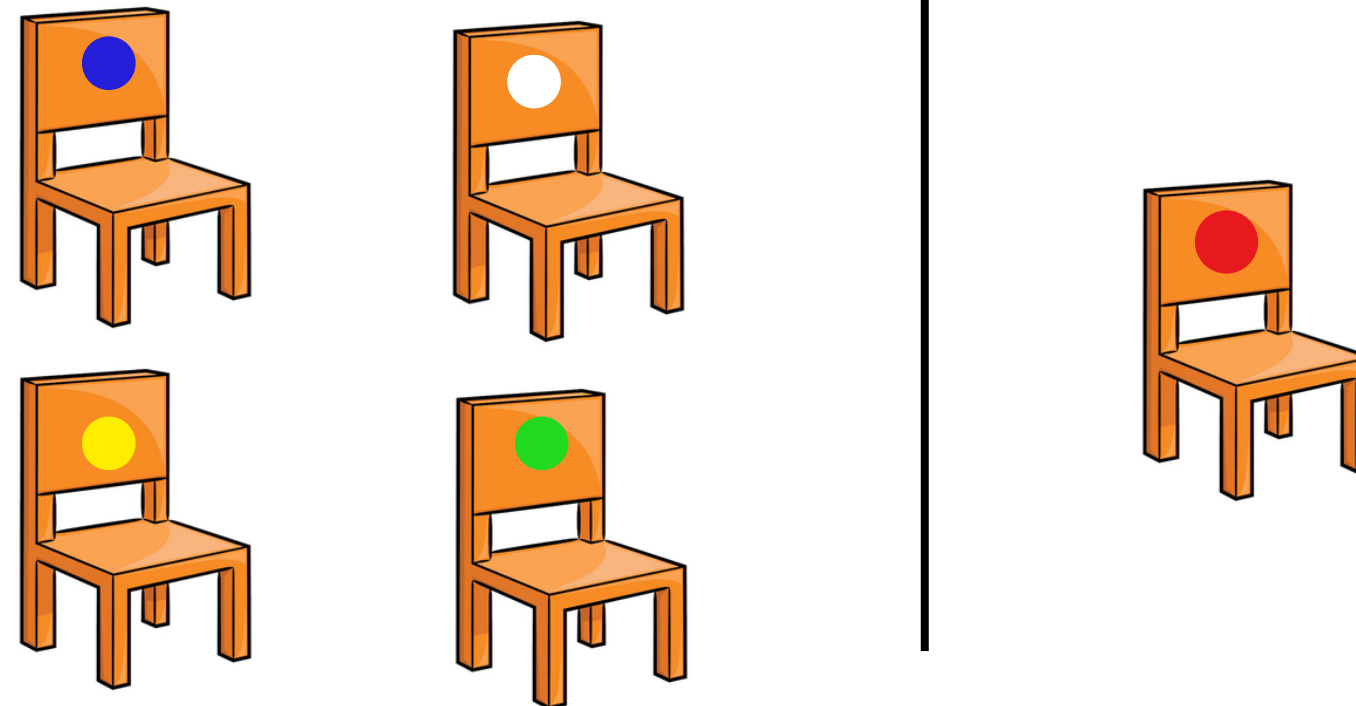
Exemplo



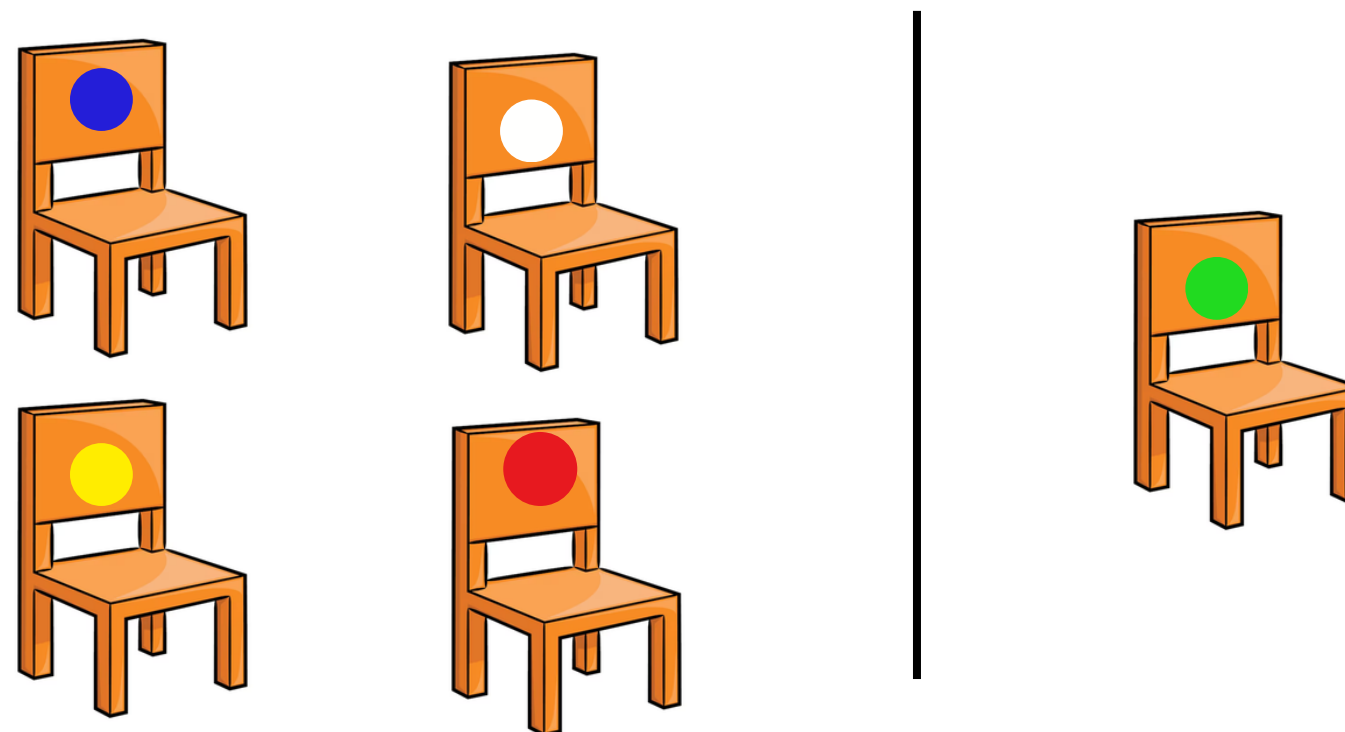
A': rotaciona as cadeiras da sala 1 no sentido anti-horário



Exemplo

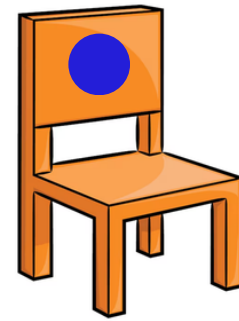


B': destroca a cadeira da sala 2 com a cadeira do canto da sala 1

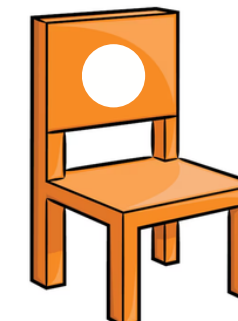
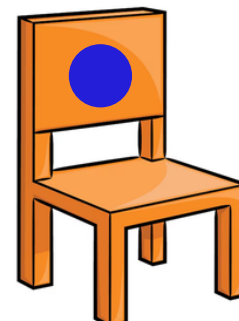


Exemplo

Estado
Inicial



Estado
Final



Comutadores na prática

A: U'

B: R' D' R

A': U

B': R' D R

Comutador: [U' R' D' R U R' D R]

Twizzle Editor ALPHA

This alg uses caret NISS notation, a convention for Fewest Moves.

 twizzle.net

Exercício (Desafio)

Embaralhe o cubo com a seguinte sequencia de movimentos:

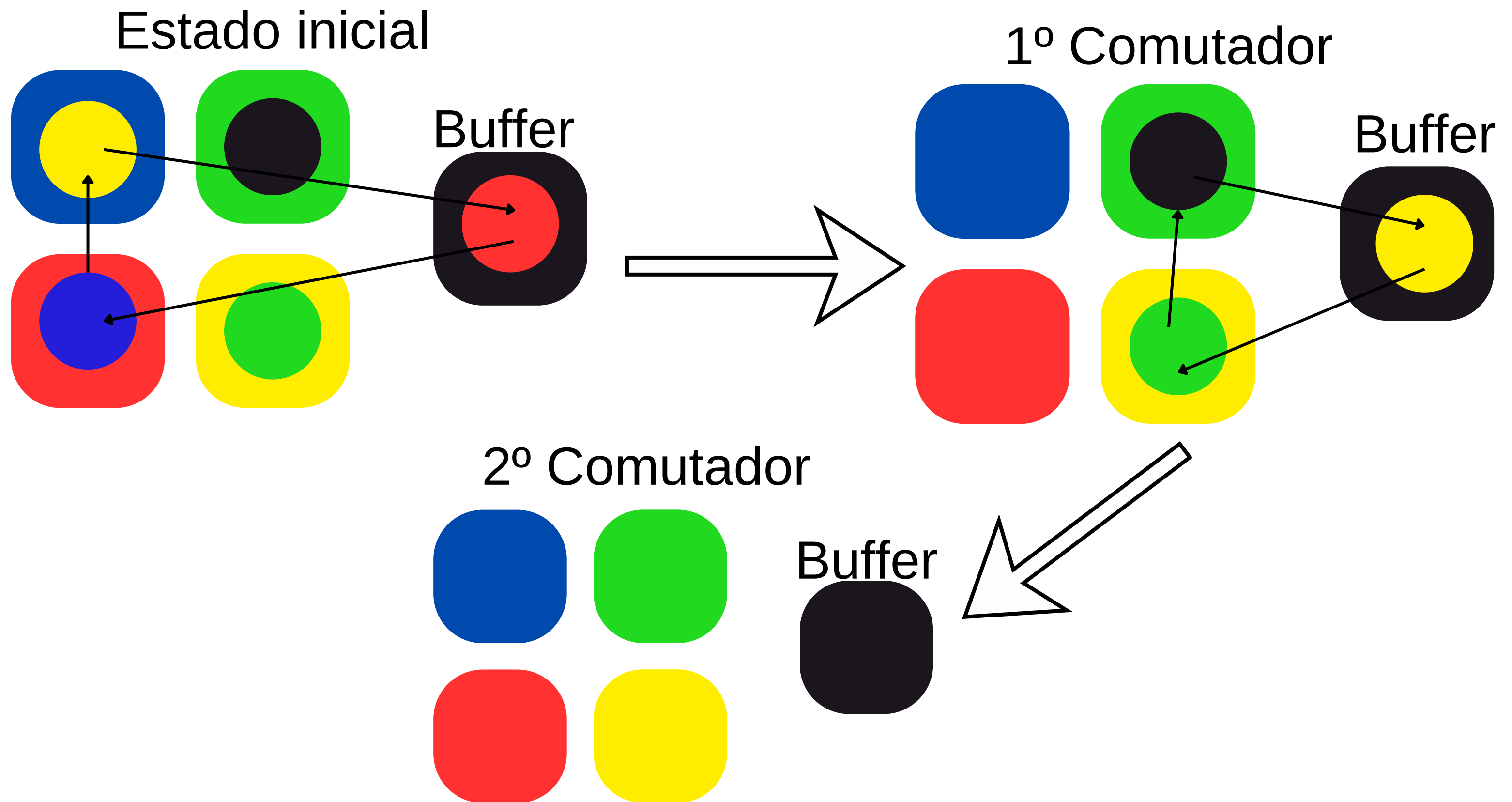
F2 D' L2 D' L2 D2 F2 U' R2 U2 L' U R2 U' L U'

Tente achar um comutador que resolva esse caso

Twizzle Editor ALPHA

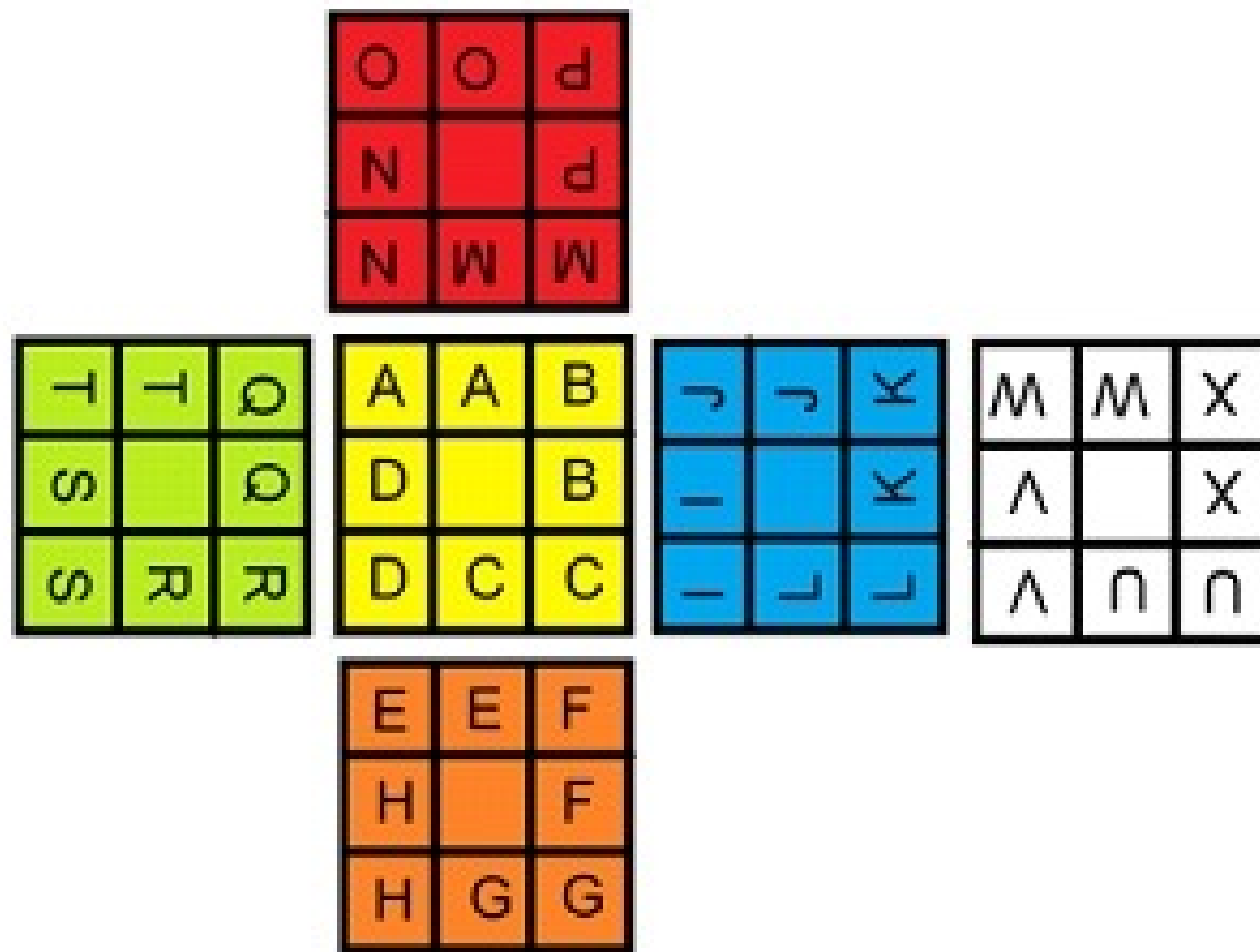
This alg uses caret NISS notation, a convention for Fewest Moves.

 twizzle.net



Sequência de solução: Vermelho; Azul; Amarelo; Verde

Como conseguir essa sequência?



História dos Meios:
[T, E, H, Q, B, L, J, B, G,
S, G, O, V]

História das Pontas:
[W, P, X, G, B, F, I]

Código em Python


```

1  inc += 2
    # Se a peça atual for considerada "certa" ou "flipada", troca o buffer por outra peça
    if (piece in certas) or (piece in flips):
        piece = solved_edges_list[inc][0]
    else:
        ready = True
    return (piece)

def adicionar_flips_1(flips, solved_edges_list, hist_edges):
    """
    Quando detectamos edges flipadas (flips), precisamos adicionar as letras
    correspondentes à história. Esta função percorre a lista 'flips' e, para cada
    peça flipada, localiza a letra correspondente em solved_edges_list e a concatena
    a hist_edges.

    Parâmetros:
    - flips: lista de edges flipadas.
    - solved_edges_list: tabela com mapeamento.
    - hist_edges: string/lista acumuladora de história.
    """
    for piece in flips:
        for j in range(len(solved_edges_list)):
            if solved_edges_list[j][0] == piece or solved_edges_list[j][0] == piece[::-1]:
                hist_edges += solved_edges_list[j][1]
    return hist_edges

# As posições dos stickers de edges no string `state` (índices onde estão os stickers)
edges = [1, 3, 5, 7, 10, 12, 14, 16, 19, 21, 23, 25, 28, 30, 32, 34, 37, 39, 41, 43, 46, 48, 50, 52]
edges_list = []
for i in range(len(edges)):
    edges_list += [state[edges[i]]]

# Tabela de edges resolvidas: cada par [nome_da_aresta, letra] mapeia a aresta à letra do diagrama.
solved_edges_list = [
    ['UB', 'A'], ['UR', 'B'], ['UF', 'C'], ['UL', 'D'],

```

Vamos testar nosso código?

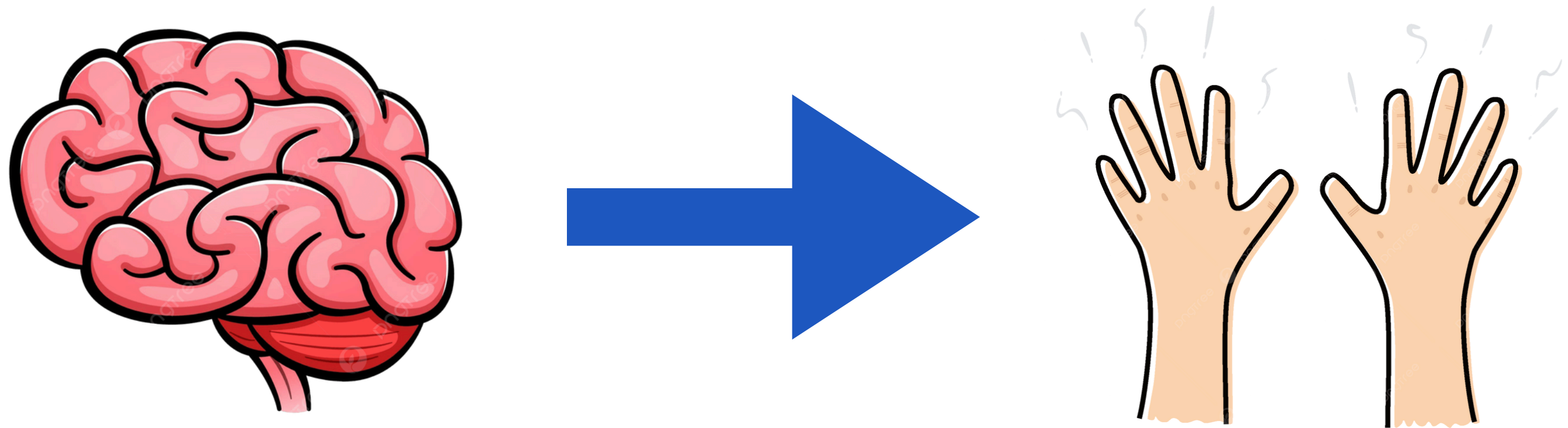
Podemos testar se a nossa solução realmente funciona através do site:

Twizzle Editor ALPHA

This alg uses caret NISS notation, a convention for Fewest Moves.

 twizzle.net

Protocolo de Saída (Cérebro → Braços)

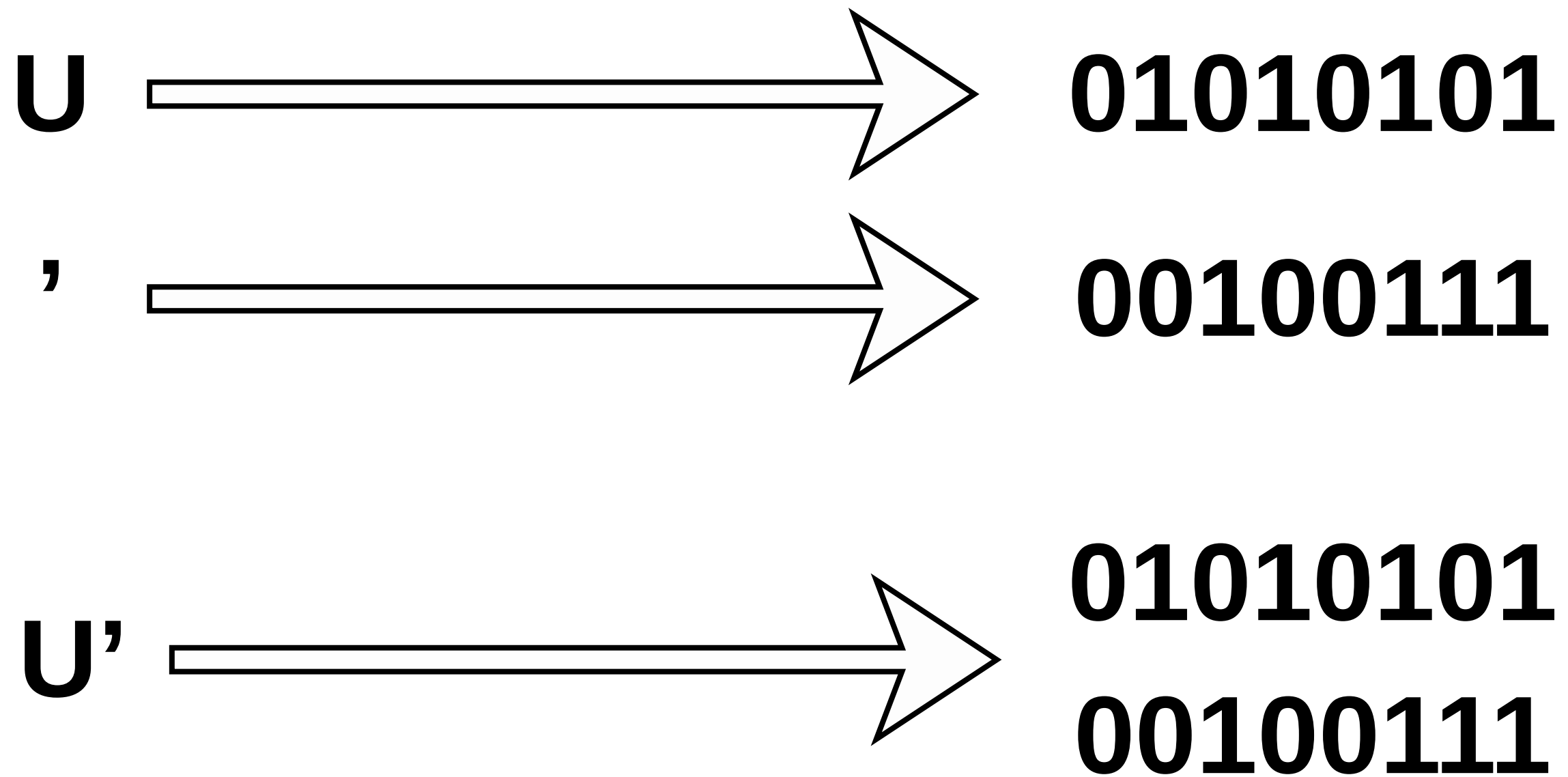


ASCII/UTF-8

Comunicação por Bytes

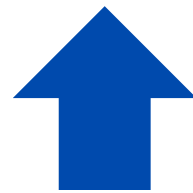
ASCII value	Char-acter	ASCII value	Char-acter	ASCII value	Char-acter	ASCII value	Char-acter	ASCII value	Char-acter	ASCII value	Char-acter
32	space	48	0	64	@	80	P	96	~	112	p
33	!	49	1	65	A	81	Q	97	a	113	q
34	"	50	2	66	B	82	R	98	b	114	r
35	#	51	3	67	C	83	S	99	c	115	s
36	\$	52	4	68	D	84	T	100	d	116	t
37	%	53	5	69	E	85	U	101	e	117	u
38	&	54	6	70	F	86	V	102	f	118	v
39	'	55	7	71	G	87	W	103	g	119	w
40	(	56	8	72	H	88	X	104	h	120	x
41	)	57	9	73	I	89	Y	105	i	121	y
42	*	58	:	74	J	90	Z	106	j	122	z
43	+	59	;	75	K	91	[	107	k	123	{
44	,	60	<	76	L	92	\	108	l	124	
45	-	61	=	77	M	93	]	109	m	125	}
46	.	62	>	78	N	94	^	110	n	126	~
47	/	63	?	79	O	95	_	111	o	127	DEL

Exemplo

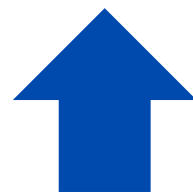


Resumo

Arduíno



$[U' R' U' R \dots R' F' R U]$

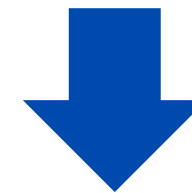


$[1000101\dots010110]$

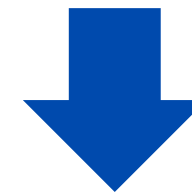
Baud Rate
Start Bit
Stop Bit
Clock

Python

Calcula a Solução



$[U' R' U' R \dots R' F' R U]$



$[1000101\dots010110]$

0100001110...



Serial